



university of
 groningen

faculty of science
and engineering

Integrating Environmental Sustainability into Process Execution Engines

Simeon Stoyanov



**university of
 groningen**

**faculty of science
and engineering**

University of Groningen

Integrating Environmental Sustainability into Process Execution Engines

Master's Thesis

To fulfill the requirements for the degree of
Master of Science in Computing Science
at the University of Groningen under the supervision of
D. (Dimka) Karastoyanova, Prof. (Bernoulli Institute, University of Groningen)
and
V. (Vasilios) Andrikopoulos, Prof. Dr. (Bernoulli Institute, University of Groningen)

Simeon Stoyanov (s6064698)

July 24, 2026

Abstract

Environmental indicators can be attached to process models during modelling, yet these annotations do not by themselves connect to a running process. This thesis investigates how modelling-level key environmental indicators (KEIs) can be used during the execution and monitoring phase of Green business process management (Green BPM) without requiring a workflow engine to interpret their environmental meaning.

The realised prototype parses BPMN4ES annotations before deploying a model to Camunda 8 and registers the extracted metadata with the identifiers assigned during deployment. During execution, a job worker reports the execution context and mock resource-use facts for configured service tasks. These facts are associated with the registered annotations and transformed into KEI-specific calculation requests. Calculation, optional target evaluation, persistence, and presentation take place outside the workflow engine, with the downstream monitoring stages connected through asynchronous messaging.

The prototype was evaluated against its functional and quality requirements using an annotated automobile-assembly model. Seventeen of eighteen functional requirements were satisfied, while execution with multiple distinct KEIs was not evaluated. Three of four quality requirements were satisfied through messaging, outage, and diagnostic assessments. Calculation modifiability was not evaluated because no second calculator was implemented. The demonstrated path covers annotated-model deployment and registration, service-task observation, carbon-emission calculation, target evaluation, persisted monitoring records, and visible detected violations.

The results show that BPMN4ES annotations can support runtime environmental monitoring alongside the normal deployment and execution path of a workflow engine. The evidence is limited to Camunda 8, one worker-observed service-task path, one carbon-emissions calculator, mock resource-use values, and reference data configured for the example. It therefore supports the runtime integration approach without establishing engine portability or the environmental accuracy of an industrial process.

Contents

1	Introduction	5
2	Background and Related Work	7
2.1	Green BPM and Environmental Process Indicators	7
2.2	Green BPM Life Cycle and Reference Architecture	8
2.3	Environmental Process Modelling and Business Process Model and Notation (BPMN) Extensions	10
2.4	Process-Level Environmental Impact Calculation	11
2.5	Runtime Monitoring and Execution-Time Integration	12
2.6	Positioning of This Thesis	13
3	Methodology	15
3.1	Approach	15
3.2	Scope	15
3.3	Formulation of Requirements	17
3.4	Architectural Constraints	17
3.5	Functional Requirements	18
3.6	Quality Requirements	19
3.7	Quality Attribute Scenarios	20
3.8	Evaluation Method	22
4	Architecture	23
4.1	Architectural Drivers	23
4.2	System Context	24
4.3	Architectural Design Decisions	26
4.3.1	Architectural design decision (ADD)-001: Parse Model Annotations and Register an External Runtime Mapping	26
4.3.2	ADD-002: Isolate Engine Integration and Observe Configured Tasks Through a Worker	27
4.3.3	ADD-003: Decouple Downstream Monitoring Through Asynchronous Messaging	28
4.3.4	ADD-004: Enrich Execution Facts Before Calculation	29
4.3.5	ADD-005: Separate Responsibilities into Deployable Services	30
4.3.6	ADD-006: Route Calculation Requests by KEI Identifier	30
4.3.7	ADD-007: Represent Service Interactions Through Explicit Compatible Contracts	31
4.3.8	ADD-008: Persist Activity-Level Results and Derive Violations	32
4.4	Architecture Views	33
4.4.1	Container View	33
4.4.2	Process Registry Component View	35
4.4.3	Monitoring Results Component View	37
4.5	Deployment and Runtime Flows	39

4.5.1	Deployment Flow	39
4.5.2	Runtime Monitoring Flow	41
4.6	Communication Strategy	43
5	Solution Realisation	44
5.1	Implementation Organisation and Deployment	44
5.2	BPMN4ES Model Translation	46
5.3	Worker Integration and Mock Operational Service	48
5.4	Observation Enrichment	49
5.5	Service-Local Contracts and Event Routing	50
5.6	Carbon-Emission Calculator	52
5.7	Target Evaluation, Persistence, and Presentation	54
5.7.1	User Task Interaction	55
5.7.2	Prototype Interface Evidence	56
6	Discussion	61
6.1	Functional Requirement Evaluation	61
6.2	Quality Requirement Evaluation	63
6.3	Answers to the Research Questions	64
6.3.1	RQ1: Transforming BPMN4ES Metadata for Runtime Use	64
6.3.2	RQ2: Providing Runtime Data from Process Execution	64
6.3.3	RQ3: Calculating KEIs from Runtime Data	64
6.3.4	Main Research Question	65
6.4	Limitations	65
6.4.1	Environmental Credibility	65
6.4.2	Runtime Coverage	65
6.4.3	Evaluation and Generalisation	65
6.5	Future Work	66
6.5.1	Measured Operational Data and Lifecycle Timestamps	66
6.5.2	Additional Observation and Calculation Paths	66
6.5.3	Returning Monitoring Results to Process Execution	66
6.5.4	Complete Quality Evaluation	67
6.6	Discussion Summary	67
7	Conclusion	68

List of Figures

2.1	Reference architecture for integrating environmental sustainability monitoring into Business process management (BPM) systems, reproduced from Medema et al. [5].	9
4.1	System context of the Environmental Monitoring System	25
4.2	Containers and neighboring runtime systems of the Environmental Sustainability Monitoring System	34
4.3	Process Registry components and external boundaries	36
4.4	Monitoring Results components, stores, and presentation boundaries	38
4.5	Deployment and registration sequence	40
4.6	Runtime observation, calculation, evaluation, and monitoring-result sequence . .	42
5.1	Annotated automobile assembly process used by the prototype	56
5.2	Frontend controls for deployment and process interaction	57
5.3	Dashboard summary and latest persisted monitoring records	58
5.4	Process-instance inspection with BPMN element overlays	59
5.5	Expanded activity-level monitoring record	59
5.6	Query and presentation of detected target violations	60
5.7	Live calculation, evaluation, and violation notifications	60

List of Tables

3.1	Architectural constraints	17
3.2	Functional requirements	18
3.3	Quality requirements	19
3.4	Quality attribute scenarios	21
4.1	Architectural drivers and implications	23
5.1	Repository units and implementation form	44
5.2	Common backend module and package structure	45
5.3	Local container deployment	45
5.4	Service-local message representations	50
5.5	Configured RabbitMQ exchanges, routing keys, and queues	50
5.6	Frontend live-update topics	55
6.1	Functional requirement evaluation	61
6.2	Quality requirement evaluation	63

Chapter 1. Introduction

Environmental sustainability is an increasingly important concern for organizations because business operations depend on natural resources and can produce negative environmental effects. Sustainability is commonly understood as meeting present needs without compromising the ability of future generations to meet their own needs, while environmental sustainability focuses more specifically on preserving ecosystems and using natural resources prudently [1], [2], [3]. In business contexts, this broader understanding implies that organizations should look beyond economic performance and consider the environmental and social consequences of their activities. This perspective appears in the concept of the triple bottom line, which frames organizational performance across economic, environmental, and social dimensions [4], [5].

These sustainability concerns are directly relevant to business processes. The execution of process activities requires inputs such as energy, materials, and water, and may produce outputs such as emissions, waste, or other ecological effects. Process-oriented sustainability research argues that environmental impact can be analysed at the level of activities and processes, rather than only at the level of entire organizations or products [2], [6], [7]. This level of analysis is important because it helps identify where environmental impact occurs and where improvement measures may target process execution. Measuring environmental impact can therefore support organisational reporting, process analysis, redesign, and operational decision-making [2], [3].

Business process management (BPM) provides concepts, methods, and techniques for modelling, executing, monitoring, analysing, and improving business processes [5]. Traditionally, BPM focuses on operational and economic process performance, commonly expressed through dimensions such as time, cost, quality, and flexibility. These dimensions are monitored through key performance indicators (KPIs), which allow organizations to evaluate whether their processes meet business objectives and where improvement is required [5], [8]. However, these traditional performance dimensions do not directly express environmental impact.

Green business process management (Green BPM) extends this perspective by treating environmental sustainability as an additional process-level concern. In this context, key environmental indicators (KEIs) describe measurable environmental aspects of process execution, such as energy consumption, greenhouse gas (GHG) emissions, water use, or waste generation [5], [8]. KEIs provide an environmental counterpart to conventional KPIs: while KPIs express business performance, KEIs express ecological performance. This makes environmental impact part of process analysis and monitoring rather than only a separate organizational reporting activity.

Prior research offers several foundational contributions to sustainability-aware process management. Studies on process modelling explore how environmental indicators can be systematically attached to Business Process Model and Notation (BPMN) process models [5], enabling the representation of environmental aspects at the level of activities and processes. In addition, efforts towards calculating environmental impact investigate how process-related data can be used to derive quantitative environmental values, such as carbon emissions, through the use of activity data and emission factors [7], [9]. These contributions show that environmental concerns can be represented in process models and that environmental values can be calculated from activity-related data.

The practical problem addressed in this thesis lies between defining environmental metadata during process modelling and using that metadata during process execution. Although a model

can state which environmental indicators apply to an activity, runtime monitoring must correlate those annotations with a particular execution and the operational data required for calculation. This thesis examines how that connection can be realised while keeping environmental processing outside the workflow engine’s native control-flow semantics.

The realised prototype transforms BPMN-level KEI metadata into a runtime representation, associates it with observed process activity execution, and processes the resulting calculation requests asynchronously. The monitoring pipeline supports environmental impact calculation, optional target evaluation, persistence, and inspection.

The main research question of this thesis is:

How can KEIs defined at the modelling level be used during process execution in BPM systems to support environmental monitoring and evaluation?

The question is split into the following sub-questions:

1. RQ1: How can KEIs defined in BPMN4ES models be transformed into a representation suitable for runtime interpretation in a BPM system?
2. RQ2: How can BPM process execution provide the runtime data needed to support the monitoring of KEIs?
3. RQ3: How can KEI values be calculated from process execution data, and what data is required to support such calculations?

The thesis contributes an architecture and a realised prototype for carrying modelling-level KEIs into runtime monitoring. The prototype demonstrates deployment-time registration, configured service-task observation, carbon-emission calculation, optional target evaluation, and inspectable monitoring records. The implementation and evaluation cover Camunda 8, one calculator, and resource-use data supplied by a mock operational service. Broader applicability and environmental accuracy therefore remain outside the demonstrated results.

The remainder of the thesis is structured as follows. Chapter 2 introduces the background and related work and positions this thesis in relation to existing Green BPM modelling, calculation, and runtime-integration approaches. Chapter 3 presents the methodology, including the research approach, scope, requirements, and evaluation method. Chapter 4 presents the architecture of the prototype. Chapter 5 describes the solution realisation. Chapter 6 evaluates the requirements, answers the research questions, and discusses the limitations and directions for future work. Chapter 7 concludes the thesis.

Chapter 2. Background and Related Work

2.1 Green BPM and Environmental Process Indicators

Green BPM extends traditional BPM by introducing environmental sustainability as a process-level concern. Conventional BPM has historically focused on economic and operational performance dimensions such as time, cost, quality, and flexibility. Seidel, Recker, and vom Broeke describe this extension as a movement from the traditional ‘devil’s quadrangle’ toward a ‘devil’s pentagon’, where sustainability becomes an additional performance dimension alongside the established process-performance concerns [10]. This framing is important because it positions sustainability not as an external reporting problem, but as a dimension of process management itself.

Several definitions of Green BPM emphasize this process-oriented character. van den Broek describes Green BPM, relying on earlier work by Opitz, Krüp, and Kolbe, as the set of information-system-supported management activities that help monitor and reduce the environmental impact of business processes during design, improvement, implementation, or operation [3]. Nowak, Leymann, and Schumm similarly frame Green BPM as a rethinking of conventional BPM from an environmental perspective, focusing on the differences and commonalities between conventional and environmentally oriented process management [8]. More recent work by Medema et al. further develops this direction by proposing a Green BPM life cycle that explicitly covers modelling, execution and monitoring, analysis, and adaptation [5]. Together, these works establish Green BPM as a life-cycle-oriented extension of BPM rather than a single modelling or reporting technique.

The relevance of environmental sustainability at the process level follows from the role of business processes in organizational activity. Medema et al. argue that business processes constitute the core of organizations and represent one of their principal sources of environmental impact [5]. Recker, Rosemann, and Gohar similarly argue that environmentally relevant actions manifest in the execution of organizational business processes and propose analysing carbon emissions at the level of process execution [7]. This process-level perspective is also supported by sustainability-oriented process analysis work, which treats the process perspective as a way to identify where environmental impacts are caused and where reductions can be implemented [2]. These contributions justify analysing environmental sustainability not only at the organizational or product level, but also at the level of process activities and process instances.

To make environmental process performance measurable, Green BPM introduces environmental indicators. Nowak, Leymann, and Schumm use the term Key Ecological Indicator and define it in relation to an ecological characteristic and a target value function [8]. Medema et al. use the term KEI and describe such indicators as a means to quantify the environmental performance of a business process [5]. The terminology differs slightly across the literature, but the underlying role is consistent: these indicators provide measurable representations of environmental performance. They therefore act as environmental counterparts to traditional KPIs. Whereas KPIs capture conventional process-performance dimensions such as time, cost, and quality, KEIs capture ecological characteristics such as resource consumption or emissions [8].

The literature also identifies recurring categories of environmental indicators. van den Broek

groups environmental process indicators into categories such as energy, emissions, waste, water, and others [3]. Willmann similarly summarizes common KEI categories as energy, water, emissions, and waste [11]. These categories are relevant because they show that environmental process monitoring is not limited to one metric such as carbon emissions. Instead, KEIs can represent several forms of ecological impact, depending on the process domain and the available data.

This body of work provides the conceptual basis for integrating environmental sustainability into process management. It establishes that environmental sustainability can be treated as a process-performance concern, that environmental impact can be analysed at the process level, and that KEIs can be used to express ecological performance in a measurable form. However, these concepts also raise a further problem: once environmental indicators are defined for business processes, they must still be connected to process execution if they are to support run-time monitoring, calculation, and evaluation. The following sections therefore examine work on environmental process modelling, process-level environmental calculation, and execution-time support for sustainability-aware processes.

2.2 Green BPM Life Cycle and Reference Architecture

A central contribution in recent Green BPM research is the proposal of a life-cycle view that integrates environmental sustainability across multiple phases of process management. Earlier work already argued that Green BPM should be understood in relation to the full business process life cycle, including design and analysis, configuration, enactment, and evaluation [8]. Medema et al. [5] refine this view into a Green BPM life cycle consisting of modelling, execution and monitoring, analysis, and adaptation. This life-cycle framing is important because it treats sustainability as a concern that must be carried from process design into execution and later improvement, rather than as an isolated modelling or reporting activity.

The motivation for such a life-cycle approach is that existing Green BPM initiatives often cover only parts of the process life cycle or focus on specific environmental indicators. This partial coverage limits an organization's ability to understand and control the environmental impact of its business processes during execution [5]. Earlier Green BPM pattern work similarly observes that conventional process optimisation approaches do not explicitly address the ecological dimension [6]. The Green BPM life cycle therefore provides a structure for identifying which concepts and tools are needed at different stages of sustainability-aware process management.

In the modelling phase, the main enabling concept is the representation of environmental indicators in process models. Medema et al. [5] introduce an extension of BPMN that allows process models to capture environmental performance indicators together with optional target values. This provides a modelling-level mechanism for associating environmental concerns with process activities. Related sustainability-oriented process analysis work uses similar ideas under different terminology, such as environmental cost drivers attached to individual activities [2]. Across these approaches, the common idea is that environmental information must be connected to process structure before it can support later analysis or execution-time use.

For the execution and monitoring phase, Medema et al. [5] propose a reference architecture that separates environmental sustainability monitoring from the BPM enactment system. As shown in Fig. 2.1, process life-cycle events are passed from the enactment system to a dedicated monitoring component. This component is responsible for environmental performance calculation, compliance checking, and predictive monitoring, while interacting with external services and information sources. The architectural significance of this structure is that sustainability monitoring is positioned around the enactment system rather than embedded directly inside the core process execution mechanism.

Within this architecture, calculator services appear as external components used by the monitoring layer to determine environmental metrics. This positioning is important because

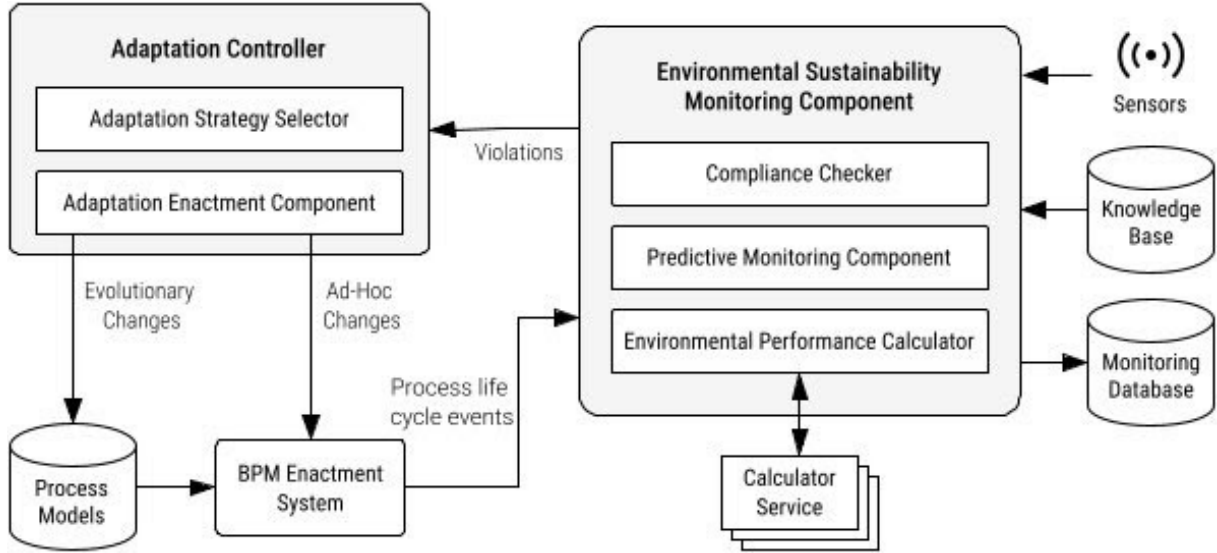


Figure 2.1: Reference architecture for integrating environmental sustainability monitoring into BPM systems, reproduced from Medema et al. [5].

it suggests that environmental calculation can be connected to process execution data without making the enactment system itself responsible for environmental calculation. Medema et al. describe this separation as a way to minimize changes to existing BPM systems while allowing calculator services to be reused in other contexts, such as process simulation or process mining [5]. Earlier Green BPM work similarly identifies the need to correlate ecological metrics with business-process runtime information [8]. The specific role of calculator services is discussed in more detail in the later section on process-level environmental impact calculation.

The later phases of the Green BPM life cycle concern analysis and adaptation. In this part of the literature, calculated or monitored environmental indicators are used to identify violations, evaluate process performance, and support improvement actions. Green business process patterns provide reusable strategies for environmentally oriented process design and adaptation [6], [12]. The life-cycle approach further discusses adaptation based on detected or predicted violations of environmental indicators, including actions such as resource selection or task scheduling [5]. These contributions show that runtime monitoring is not an end in itself in the broader Green BPM vision; rather, it is intended to provide the information needed for later analysis and adaptation.

At the same time, the literature identifies important limitations in existing Green BPM support. Many initiatives focus on individual lifecycle phases or specific indicators, and available prototypes do not yet provide the full functionality required across the complete Green BPM life cycle [5]. Related work also highlights practical challenges in data availability for exact sustainability assessment, since many approaches assume that the required environmental data is already available in organisational systems [13]. These limitations are important because they show that the main challenge is not only defining environmental indicators, but also connecting them to the data and runtime mechanisms needed for monitoring and calculation.

The Green BPM life cycle and reference architecture therefore provide the main conceptual foundation for execution-time sustainability support. They establish the need to carry environmental indicators from modelling into execution and monitoring, and they motivate the use of monitoring components and external calculation services rather than embedding all sustainability logic directly into a process engine. The following sections examine the modelling and calculation sides of this problem in more detail.

2.3 Environmental Process Modelling and BPMN Extensions

A recurring concern in Green BPM research is how environmental information can be represented directly in process models. Modelling-level representation is useful because process models are not only technical artefacts, but also communication and analysis artefacts. They make activities, resources, and dependencies visible to process stakeholders. In the sustainability context, this visibility can support the identification of activities with high environmental impact and provide a basis for later analysis or redesign [3], [7]. However, standard BPMN does not provide native modelling constructs for environmental indicators, which motivates the use of BPMN extensions or additional annotations for sustainability-related information [5].

Earlier modelling approaches attempted to represent environmental concerns by adding specific sustainability-related annotations to process models. Work focused on carbon footprint introduced notational support for marking activities associated with paper or fuel consumption and for visualising carbon-related information in the process model [7]. Other sustainability-oriented approaches use activity-level concepts such as environmental cost drivers to connect process activities with environmental impact information [2]. These approaches show that the activity level is a natural place for environmental annotations, because process-level environmental impact is usually produced through the execution of concrete activities rather than by the process model as an abstract whole.

The BPMN4ES approach introduced by Medema et al. [5] generalises this idea by extending BPMN with KEI metadata. Instead of focusing only on one environmental metric, such as carbon emissions, BPMN4ES allows different environmental indicators to be associated with BPMN activities. The extension defines environmental indicators with attributes such as unit and optional target value, and attaches them to activity elements through a dedicated container structure. This is relevant because it allows the model to express not only that an activity has an environmental concern, but also which indicator should be monitored and which value may serve as a threshold during later evaluation.

An important aspect of BPMN4ES is that environmental information is intended to remain both visible to modellers and machine-readable for tools. Medema et al. [5] describe the use of the BPMN extension mechanism to store environmental metadata in the BPMN Extensible Markup Language (XML) representation while preserving the standard visual appearance of BPMN elements. The BPMN4ES repository provides the corresponding modelling-tool artefact as an extension of `bpmn-js`, allowing the notation to be used in a graphical modelling environment [14]. The paper and the tool artefact therefore jointly establish BPMN4ES as both a modelling notation and a machine-readable representation of environmental metadata.

Subsequent work in the same research line builds on this BPMN4ES modelling tool rather than proposing a separate environmental modelling language. Willmann [11] discusses the use of BPMN4ES-style annotations for attaching KEIs to activity elements with optional target values and visual indicators. Rasmussen [15] provides implementation-oriented detail on how such metadata appears in BPMN XML, including fields such as indicator identity, unit, target value, and icon. These works are useful here mainly because they show how the BPMN4ES modelling artefact is used and interpreted in later prototype work within the same research line.

At the same time, modelling-level environmental indicators have an inherent limitation when considered in isolation. They can specify what should be monitored, but they do not by themselves calculate environmental impact or observe actual process execution. Medema et al. [5] identify this broader issue by arguing that Green BPM initiatives often focus on individual lifecycle phases and that additional support is needed to control environmental impact during execution. This limitation is also visible in later prototype work from the same research line, where BPMN4ES-style metadata must still be interpreted or connected to execution infrastructure before it can support runtime tracking [11], [15]. This means that modelling-level annotations are necessary but not sufficient: they provide environmental metadata, but further

runtime and calculation mechanisms are required to use this metadata during process execution.

Overall, environmental process modelling work establishes the modelling-side foundation for sustainability-aware process execution. It shows how environmental indicators can be attached to process activities, how they can be represented as structured metadata, and how modelling tools can make them visible to process designers. The remaining challenge is to connect this modelling-level information to execution-time data and environmental calculation. This connection is addressed by calculator-oriented and runtime-oriented approaches discussed in the following sections.

2.4 Process-Level Environmental Impact Calculation

Beyond modelling environmental indicators, related work also addresses the calculation of environmental impact at the level of process activities and process instances. This perspective is grounded in the idea that the environmental impact of a business process stems from the tasks, subprocesses, and resources involved in its execution [5]. Recker, Rosemann, and Gohar [7] similarly argue that environmentally relevant actions manifest in the execution of organizational business processes. Process-level calculation is therefore useful because it allows environmental impact to be analysed where operational activities actually occur, rather than only at the level of aggregate organizational reporting.

A common way to calculate environmental impact at this level is to use an activity-based or bottom-up approach. Recker et al. [7] introduce Activity-Based Emission analysis as an extension of Activity-Based Costing, where environmental measures are connected to process activities and resources. van den Broek [3] describes bottom-up calculation as an approach that uses known consumption of atomic resources to derive the consumption attributable to tasks. Popescu [9] applies the same general principle in a calculator-service context, where task-specific information and equipment usage are used to estimate emissions at a granular level. The common assumption across these approaches is that process-level impact can be constructed from activity-level contributions.

The basic calculation pattern relies on combining a quantity of resource use with an environmental conversion factor. In emissions accounting, emission factors convert an activity, fuel, energy, or material quantity into an estimated amount of emitted greenhouse gas. Popescu [9] defines emission factors as coefficients that express average greenhouse gas emissions per unit of activity, consumed energy, or material quantity. In his calculator, this principle is applied to fuel-based CO₂ calculation, as shown in Equation 2.1.

$$E_{CO_2} = FQ \times EF \quad (2.1)$$

where E_{CO_2} denotes the estimated CO₂ emissions, FQ denotes the fuel quantity, and EF denotes the emission factor.

For business processes, activity data must be connected to the execution of specific activities. Recker et al. [7] describe the calculation of carbon emissions by gathering activity data for each process and resource. In calculator-oriented work, this can involve combining runtime task duration with resource metadata. For example, if a task uses a resource for a known duration and the resource has a known fuel or energy consumption rate, the consumed amount can be derived from the duration and consumption rate before applying the relevant emission factor [9]. Later prototype work in the same research line also uses this type of duration-and-resource-based calculation, where task duration is combined with fuel usage per time unit [15].

After activity-level values have been calculated, they can be aggregated to obtain a process-level result. Recker et al. [7] describe the overall carbon footprint of a process as the sum of emissions across its activities and emission scopes. van den Broek [3] provides a more process-structure-oriented view by discussing how activity emissions can be summed across paths in a process model. In the calculator service discussed by Medema et al. [5], the total carbon

emissions of a process instance are calculated by summing the emissions of its activities when the process instance ends. Aggregation therefore turns local activity calculations into process-instance-level environmental results.

These calculation approaches also involve assumptions and limitations. Popescu’s calculator demonstrates a practical bottom-up calculation model, but its scope is deliberately limited to CO₂ emissions and to resource configurations for which activity-level consumption can be estimated [9]. In particular, the approach is strongest for atomic resources with known average consumption rates, while shared resources, such as heating or lighting systems used by several activities, are more difficult to allocate to individual tasks. Medema et al. [5] also note that this kind of calculation may rely on constant resource consumption rates, which simplifies real execution behaviour. These limitations show that process-level environmental calculation is an estimation mechanism whose accuracy depends on the granularity and reliability of the available execution data, resource metadata, and emission factors.

Popescu’s environmental sustainability calculator is especially relevant as a calculator-oriented precedent. It demonstrates that carbon-emission calculation can be implemented as an external service around business process execution, using process-related data, resource information, and emission factors to calculate CO₂ emissions [9]. Medema et al. [5] position this type of calculator service within a broader Green BPM architecture, where external services are connected to process life-cycle events through monitoring components. The important contribution of this line of work is that environmental calculation does not need to be embedded directly into the enactment system. Instead, process execution can provide the contextual and activity-level data needed by an external calculator.

This line of work provides the calculation-side foundation for sustainability-aware process execution. It shows how environmental values can be estimated from activity data, resource metadata, and emission factors, but also shows that calculation depends on receiving suitable execution context and environmental metadata.

2.5 Runtime Monitoring and Execution-Time Integration

The previous sections show that environmental indicators can be represented at the modelling level and that environmental values can be calculated from activity-related data. A remaining issue is how the required execution data is obtained during process execution and connected to the relevant environmental metadata. Runtime monitoring addresses this problem by observing process execution events, extracting execution context, and making this information available for environmental calculation or evaluation.

Popescu’s calculator service provides an important runtime-oriented precedent. The system is implemented around existing BPM engines, with integrations for Camunda 7 and jBPM, using an integration layer that observes process execution events and forwards them to an external calculation service [9]. In particular, Popescu describes the use of event listeners for process start, process end, activity start, and activity end events. These events provide the temporal and contextual basis for calculating emissions outside the process engine. The calculator therefore demonstrates that environmental calculation can be connected to runtime process execution without making the workflow engine itself responsible for performing the calculation.

The architectural separation in Popescu’s work is also relevant. The calculator is implemented as an external service, while integration modules act as intermediaries between the BPM engine and the main application [9]. This allows process events to be collected from the execution environment and then processed by a separate component responsible for emission calculation, persistence, and dashboard-oriented reporting. However, the environmental metadata used by the calculator is supplied through separate annotations and resource configuration rather than through BPMN-level KEI metadata. As a result, Popescu’s work demonstrates runtime environmental calculation around process execution, but it does not fully address the

problem of interpreting BPMN4ES-style environmental indicators during runtime.

Later work in the same research line moves closer to the use of BPMN-level sustainability metadata during execution. Rasmussen [15] uses BPMN4ES metadata together with Camunda 7 listener mechanisms to trigger emission calculation for completed tasks. The implementation extracts task duration and combines it with resource information such as fuel type, fuel unit, and fuel consumption rate. This provides a concrete example of how modelled sustainability metadata and runtime execution data can be connected for environmental calculation. At the same time, the work also reports a limitation in measurement accuracy, since task duration can be affected by the engine’s polling mechanism [15].

Willmann [11] also builds on BPMN4ES, but investigates the problem in the Camunda 8 ecosystem. The work includes modelling-tool support by creating a plugin for Camunda Modeler so that BPMN4ES-style annotations can be added in the modelling environment and represented in the resulting BPMN model. On the execution side, the implementation uses Zeebe execution listeners and job workers to retrieve sustainability-related metric values during or after process activity execution. One approach uses listener-triggered workers after activity completion, while another uses a polling mechanism to retrieve metric values during task execution. Willmann also discusses a decoupled retrieval approach in which a worker sends a network request to an external server that manages environmental indicators independently [11].

These runtime-oriented implementations expose important limitations. Rasmussen’s work shows that engine-level timing and polling behaviour can affect the precision of duration-based calculations [15]. Willmann’s work shows that runtime metric retrieval can introduce execution delays whose duration depends on the external metric endpoint implementation [11]. These limitations indicate that runtime monitoring is not only a matter of obtaining data, but also of deciding how closely environmental monitoring should be coupled to process execution. Monitoring mechanisms that run directly in the execution path can provide timely information, but they may also affect execution latency or depend strongly on engine-specific listener semantics.

Taken together, these works form the immediate runtime-integration context for this thesis. Popescu demonstrates that environmental calculation can be implemented as an external service connected to process execution events, but does not use BPMN-level KEI metadata as the source of environmental monitoring requirements. Rasmussen and Willmann build on the BPMN4ES research line and show how sustainability-related metadata can be connected to workflow engine execution mechanisms, but their implementations also reveal limitations related to engine-specific integration, runtime measurement precision, and execution overhead. The remaining gap is therefore not simply how to calculate emissions, but how to connect modelling-level environmental indicators, runtime process execution data, and environmental calculation in a way that preserves a clear boundary between process enactment and sustainability monitoring.

2.6 Positioning of This Thesis

The reviewed literature provides three main foundations for this thesis. Green BPM research establishes environmental sustainability as a process-level concern and introduces environmental indicators as measurable process-performance concepts. BPMN4ES provides a modelling-level mechanism for attaching environmental indicators and target values to BPMN activities. Calculator-oriented work demonstrates that environmental impact can be estimated from process-related data, resource metadata, and emission factors.

The gap addressed in this thesis lies between modelling-level environmental metadata and runtime process execution. Popescu’s calculator shows that environmental calculation can be implemented as an external service around process execution, including integrations with Camunda 7 and jBPM. However, its environmental metadata is supplied through separate annotations and resource configuration rather than through BPMN-level KEI metadata. Rasmussen and Willmann move closer to the BPMN4ES research line by connecting BPMN4ES metadata

to workflow engine execution mechanisms. Their work demonstrates that listeners, workers, and runtime retrieval mechanisms can support sustainability tracking, but also reports practical limitations such as duration-measurement imprecision, endpoint-dependent execution delays, and engine-specific implementation constraints.

This thesis is positioned in the same modelling-to-execution problem space. Its focus is the runtime connection between BPMN-level KEI metadata, observed process activity execution, and downstream environmental calculation. The implemented prototype keeps environmental calculation and threshold evaluation outside the engine’s core control-flow semantics, while still using execution context produced during process execution. The contribution is therefore not a new indicator taxonomy or a new emission factor method, but an integration approach for making modelled environmental indicators usable during runtime monitoring and calculation.

Chapter 3. Methodology

3.1 Approach

This thesis follows a design approach based on a software prototype. The problem is not investigated through a controlled empirical study, but through the design, implementation, and evaluation of a software artefact. The artefact is used to examine how environmental indicators defined at the BPMN modelling level can be made available during process execution and used for runtime monitoring, calculation, and evaluation.

The work is organised into four steps. First, academic literature is analysed to establish the conceptual foundation and the unresolved integration problem. The analysis covers Green BPM, KEIs, environmental modelling at the BPMN level, environmental impact calculation at the process level, and sustainability-oriented runtime monitoring. The conclusions from this literature analysis establish the modelling-to-execution connection addressed by the prototype.

Second, the supported integration mechanisms of the selected workflow engine ecosystem are examined through its technical documentation. The analysis considers process deployment, instance creation, jobs, workers, variables, connectors, and lifecycle listeners in Camunda 8 [16], [17], [18], [19], [20], [21], [22], [23]. The purpose of this activity is not to infer general properties of all workflow engines. It is to establish which documented boundary can expose the process, instance, and activity context needed by the prototype without modifying the selected engine. The detailed comparison and resulting selection are recorded as architectural decisions in Chapter 4.

Third, a software prototype is designed and implemented. The prototype connects KEI metadata from the modelling level, observed process activity execution, environmental calculation, threshold evaluation, and monitoring records. The constraints and requirements are stated before the architecture in this thesis to make the traceability from the research problem to the design choices explicit.

Fourth, the prototype is evaluated against the functional requirements and quality attribute scenarios. The evaluation uses the annotated BPMN model, its execution through the workflow engine, the resulting monitoring events and records, and separate assessments of service outages and processing errors. The findings are used to answer the research questions stated in Chapter 1.

3.2 Scope

The thesis is scoped to the execution and monitoring phase of the Green BPM life cycle [5]. It does not address the complete life cycle of sustainability-aware process management. The primary runtime unit of analysis is one configured service task execution associated with one or more distinct modelling-level KEIs. These service tasks are derived from an example BPMN workflow model on automobile assembly, extracted from [9]. The model is manually assigned its KEIs in the BPMN4ES web tool¹ [14]. Model deployment and registration are additionally examined for RQ1. The runtime outcome is an activity-level monitoring record, not an aggregate environmental assessment of the complete process instance.

¹<https://github.com/michel-medema/BPMN4ES>

The modelling and deployment phase of a workflow is the boundary where the initial workflow gets assigned the aforementioned annotations. The annotations identify the KEIs, the BPMN element to which they apply, the unit of the calculated environmental value, and possibly a maximum target. They represent what should be monitored, and not a measured consumption. Two steps follow modelling. First, the example workflow is deployed and the engine assigns its identifiers. Second, the deployed definition is registered for runtime monitoring. In this thesis, registration denotes the association of those engine identifiers with the original model elements and their annotations.

The runtime scope is limited to service tasks configured for the prototype’s worker job type. For each activated job, the worker receives the process definition, process instance, element instance, and BPMN-element identifiers. It invokes a deterministic mock operational service using the configured work object. In the automobile-assembly scenario, the literal work-object type is `car`. The operational service returns resource-usage facts for the completed work, which are used to calculate the KEIs registered for the corresponding BPMN element.

The prototype does not observe user tasks, gateways, events, or service tasks configured for other job types. It also does not reconstruct complete activity lifecycles, calculate authoritative activity durations, or produce an aggregated environmental assessment for the complete process instance.

Downstream processing associates the operational work facts with the annotations registered for the same process definition and BPMN element. It creates a calculation request for each distinct applicable annotation when resource usage is available. The implemented calculator covers the `carbon-emissions` KEI. It combines runtime resource usage with configured resource profiles and emission factors [9]. A numeric target, when present, is evaluated as a maximum acceptable value for the calculation. An absent target is valid and produces a calculation-only monitoring record.

The workflow engine remains responsible for process orchestration. Environmental calculation, target evaluation, result persistence, and visualisation are outside its control-flow semantics. The prototype reads resource-usage values from a JavaScript Object Notation (JSON) file and uses configured environmental reference data, allowing the same calculation inputs to be used again. It does not integrate physical sensors or claim that these data constitute a production environmental knowledge base.

The following concerns are outside the evaluated scope:

- runtime observation across all BPMN element types;
- historical process mining;
- process-instance-level KEI aggregation;
- predictive monitoring;
- automated process adaptation or remediation;
- resolution of detected violations;
- restriction of process-instance creation;
- regulatory environmental accounting;
- production-scale availability and security;
- modifications to the workflow engine;
- verified portability to a workflow engine other than Camunda 8;

The architecture defines boundaries at which some of these concerns could be extended, but such extension points are not treated as implemented or evaluated capabilities.

3.3 Formulation of Requirements

The requirements are derived from three sources. The research questions determine the required modelling-to-runtime transformation, execution context and calculation path. The reviewed literature supplies the environmental concepts and the separation between process modelling, execution, calculation, and monitoring. As described in Section 3.1, the official documentation of the selected workflow engine ecosystem was reviewed to identify its supported deployment and runtime integration mechanisms. The resulting choice of a worker-based integration mechanism and its limitations are documented in Section 4.3.2. Together, these inputs inform the architectural constraints, functional requirements, and observable quality requirements defined below.

Requirements are assigned one of three priority levels according to their significance within the prototype evaluation. *Must* denotes a capability required to answer at least one research question within the defined prototype boundary. *Should* denotes a capability that materially improves the credibility or usability of the demonstration but is not necessary to establish the main integration chain. *Could* denotes a bounded extension that is desirable but does not affect the central demonstration. Priority expresses the significance of a requirement rather than its implementation status. Satisfaction is determined separately using the evidence defined in Section 3.8.

3.4 Architectural Constraints

The constraints in Table 3.1 are fixed boundaries within which the prototype architecture is designed.

Table 3.1: Architectural constraints

ID	Constraint
AC-01	The workflow engine is treated as a black-box execution environment. The prototype may use engine-supported modelling or runtime mechanisms, but it shall not require modifications to the workflow engine or require the engine to interpret KEIs as native execution concepts.
AC-02	Annotated process models shall remain executable by the target workflow engine without requiring the engine to understand BPMN4ES extensions. From the engine’s point of view, environmental annotations must not affect normal process execution.
AC-03	The prototype targets Camunda 8 for implementation and evaluation. The architecture shall isolate engine-specific integration mechanisms so they can be extended or replaced. Full execution across different workflow engines is outside the required prototype evaluation.
AC-04	KEI definitions originate from the modelling layer. Runtime components may parse, register, and enrich this metadata, but shall not infer which process activities are environmentally relevant.
AC-05	Environmental calculation, target evaluation, result persistence and visualization shall remain outside the workflow engine’s control-flow semantics.
AC-06	Environmental calculation and threshold-based compliance evaluation shall use data available within the prototype boundary. The prototype may use process variables, execution data produced by workers, configured values, and simulated test data, but it shall not depend on live sensor integration or a production environmental knowledge base.

3.5 Functional Requirements

The functional requirements in Table 3.2 define the capabilities required of the prototype to support execution-time environmental monitoring. They are derived from the research questions, the defined scope, and the architectural constraints presented above. Some requirements can be assessed through operator-facing outputs, such as monitoring records exposed through the prototype’s application programming interface (API) or user interface. Requirements concerning intermediate transformations must instead be assessed at the corresponding service or messaging boundary.

Table 3.2: Functional requirements

ID	Priority	Requirement
FR-01	Must	The system shall support deployment of an executable BPMN process model containing modelling-level KEI metadata.
FR-02	Must	The system shall start a deployed process definition with supplied process variables.
FR-03	Must	The system shall parse KEI metadata associated with BPMN elements from the submitted model before deployment.
FR-04	Must	The system shall register a runtime mapping between the deployed process definition, its annotated BPMN elements, their distinct KEI annotations, units, and optional targets.
FR-05	Must	For each observed execution, the system shall associate the operational resource usage facts with their process definition, process instance, element instance and BPMN element when those identifiers are provided by the engine.
FR-06	Must	The system shall observe configured service task executions through a workflow engine worker and obtain resource usage facts for the completed operational work.
FR-07	Must	The system shall publish an engine-task-completed event containing service-task execution context and completed operational work facts before completing the corresponding engine job.
FR-08	Must	The system shall retrieve the registered annotations for the observed process definition and BPMN element and publish one calculation request for each distinct applicable KEI when resource usage is present.
FR-09	Must	The system shall route each calculation request according to its KEI identifier so that a calculator bound to that indicator can receive it.
FR-10	Must	The system shall calculate values for supported KEIs associated with the same BPMN activity when their calculator routes are available.
FR-11	Must	The system shall persist a completed calculation as a monitoring record when its annotation contains no target.
FR-12	Could	The system may support multiple distinct KEIs associated with the same BPMN process activity when their calculation routes are available.

ID	Priority	Requirement
FR-13	Must	When a maximum target is present, the system shall classify the calculated value as <code>WITHIN_TARGET</code> when it is less than or equal to the target and as <code>VIOLATED</code> when it exceeds the target
FR-14	Must	The system shall persist one activity-level monitoring record for a completed calculation without a target or for a completed target evaluation. The result shall include identifiers for the process definition, process instance, element instance, BPMN element, and KEI annotation, together with the calculation metadata, resource-level contributions, and evaluation outcome when present.
FR-15	Should	The system shall expose persisted monitoring records through an operator-facing API or user interface.
FR-16	Should	The system shall log missing monitoring inputs, unroutable calculation requests, invalid targets, and handled calculation failures at the component that detects them. These conditions shall not produce a completed monitoring record.
FR-17	Must	The system shall expose persisted results classified as <code>violated</code> through a queryable operator-facing interface.
FR-18	Should	The system shall expose process-instance details containing its monitoring records, detected violations, resource breakdowns and the deployed BPMN model when available.

3.6 Quality Requirements

The quality requirements in Table 3.3 define properties that influence the architecture and the credibility of the prototype evaluation.

Table 3.3: Quality requirements

ID	Title	Priority	Requirement
QR-01	Downstream Fault Isolation	Must	When operational work and publication of the completed task event succeed, the availability or processing time of downstream monitoring consumers shall not prevent the worker from completing the corresponding workflow engine job.
QR-02	Calculation Modifiability	Should	A calculator for an additional KEI that conforms to the established calculation request contract shall be introducible without source code changes to the workflow worker or changes to the BPMN-to-KEI registration schema.
QR-03	Failure Diagnosability	Should	The absence of applicable KEI metadata or resource-usage data, an unroutable calculation request, a non-numeric target value, and a handled calculation failure shall produce a service-local diagnostic log entry from which the detecting processing stage and encountered condition can be identified.
QR-04	Message Compatibility	Must	For each configured messaging route, a payload emitted by its producer shall be deserialisable and valid according to every configured consumer's message representation.

3.7 Quality Attribute Scenarios

The quality requirements are refined into quality attribute scenarios using the scenario-based specification structure proposed by Bass, Clements, and Kazman [24]. Each scenario specifies six elements: the *source of stimulus*, the entity that generates the event; the *stimulus*, the event or condition requiring a response; the *environment*, the context in which the stimulus occurs; the *artifact*, the affected part of the system; the *response*, the activity undertaken by the system or developers; and the *response measure*, the criterion used to evaluate whether the response satisfies the requirement. This structure is used to make the architectural quality requirements concrete, observable, and evaluable.

Table 3.4: Quality attribute scenarios

QR	Source	Stimulus	Environment	Artifact	Response	Response measure
QR-01	Downstream monitoring consumer	A downstream consumer becomes unavailable or delays processing.	During a configured service-task execution after the operational work and publication of the completed-task event have succeeded.	Camunda 8 Worker Service and the corresponding workflow engine job.	The worker completes the workflow engine job without calling or awaiting the downstream consumer.	The job is completed and the process advances while the downstream consumer remains unavailable or delayed, without a worker retry, failed job, or engine incident caused by the downstream condition.
QR-02	Developer	Support is requested for an additional KEI calculator that conforms to the established calculation-request contract.	At development and build time, after the original prototype has been implemented.	The calculator integration boundary, Camunda 8 Worker Service, and the registered-model schema in Monitoring Results Service.	The developer adds and configures the calculator without modifying the workflow worker or the registration schema.	The additional calculator successfully processes a representative request, while inspection of the change set shows no source-code changes to Camunda 8 Worker Service and no changes to the BPMN-to-KEI registration schema.
QR-03	Evaluator	The monitoring pipeline encounters one of the following missing-data or processing-error conditions: no applicable KEI metadata is returned, resource-usage data is absent, a calculation request has no matching route, a target value is non-numeric, calculation encounters a handled failure.	During operation of the monitoring pipeline.	Observation Service, CO2 Calculation Service, or KEI Evaluation Service, depending on where the condition is detected.	The detecting component emits a service-local diagnostic log entry describing the encountered condition.	For each condition named in the stimulus, the logger identifies both the detecting component and the encountered condition. The scenario is satisfied only if this holds for every condition.
QR-04	Producer service	The producer publishes a runtime event on a configured messaging route.	During normal execution with the messaging infrastructure and intended consumer active.	The configured messaging route, consumer message representation, and inbound messaging adapter.	Each configured consumer deserialises and validates the payload and passes the resulting message to its application processing operation.	For every configured producer-consumer pair, the emitted event is accepted without a deserialisation or validation error and its required correlation and domain values are preserved.

3.8 Evaluation Method

The functional requirements provide the baseline against which the realised prototype is evaluated. They form the backbone of its two principal flows. The first covers the deployment of an annotated workflow and the registration of its associated KEI metadata. The second starts a process instance and follows a configured service task through the collection of resource-usage data, environmental calculation, optional target evaluation, and the production of monitoring records and detected violations.

Each functional requirement is evaluated against the realized behaviour of the prototype, while each quality requirement is evaluated through the response measure of its corresponding quality attribute scenario. Both functional and quality requirements are classified as *satisfied*, *not satisfied*, or *not evaluated*. *Satisfied* means that all required behavior is demonstrated, *not satisfied* means that the relevant assessment is performed but required behavior is absent, and *not evaluated* means that the assessment needed to reach a conclusion is not performed.

Chapter 6 synthesizes the requirement-level findings to answer the research questions within the defined scope. For each research question, the discussion considers whether the collected evidence provides a substantiated answer, indicates a limitation of the realized prototype, or remains insufficient to support a conclusion.

Chapter 4. Architecture

This chapter presents the architecture developed from the evidence collected through the literature and technical-document analysis and from the constraints and requirements formulated in Chapter 3. These outputs provided the basis for separating model registration, workflow engine integration, runtime observation, environmental calculation, target evaluation, and result management. Realizing this separation introduces cross-service contracts, broker configuration, eventual consistency, and additional operational overhead. The architectural decisions explain how these trade-offs are handled within the prototype scope.

At its highest level, the resulting architecture consists of the Environmental Monitoring System, a workflow engine adapter layer, RabbitMQ, and the Frontend. The Environmental Monitoring System owns the runtime monitoring responsibilities, the adapter layer mediates interaction with Camunda 8, RabbitMQ transports downstream monitoring events, and the Frontend supports model deployment, process interaction, and result inspection. Together, these parts provide the path from annotated-model deployment and registration to runtime observation, calculation, optional target evaluation, persistence, and inspection. The following chapter describes their concrete implementation.

4.1 Architectural Drivers

Architectural drivers identify the requirements and constraints that have a more significant effect on the system structure. Table 4.1 states the structural implications that the architecture must address.

Table 4.1: Architectural drivers and implications

Driver	Source	Architectural implication
External representation of modelled metadata	AC-02 , AC-04 , FR-03 , and FR-04	Modelling-level KEI metadata must remain external to Camunda while being correlated with the identifiers assigned during deployment.
Isolated workflow engine integration	AC-01 , AC-03 , FR-01 , FR-02 , and FR-06	Engine-specific interfaces and objects must remain behind a dedicated integration boundary.
Downstream monitoring independence	AC-05 , FR-07 , and QR-01	Workflow-job completion must not depend on the availability or processing latency of downstream monitoring components.
Execution-to-calculation transformation	FR-05 , FR-08 , FR-09 , and FR-10	Runtime execution facts must be correlated with registered element-level annotations and transformed into KEI-specific calculation requests.

Driver	Source	Architectural implication
Calculator extensibility	QR-02	Additional calculators must be introducible through an established request and routing boundary without modifying workflow integration or process-model registration structures.
Persistent and inspectable monitoring records	FR-11, FR-14, FR-15, and FR-18	Calculation and optional evaluation outcomes must have an authoritative owner and remain queryable independently of workflow engine state.
Target evaluation and violation visibility	FR-13 and FR-17	Calculated values with numeric targets must be classified against those targets, and persisted results classified as VIOLATED must be retrievable through an operator-facing interface.

The architectural design decisions in the following section explain how the prototype addresses these drivers and record the resulting trade-offs.

4.2 System Context

Figure 4.1 presents the Environmental Monitoring System as the system of interest. It connects modelling-level KEI metadata with workflow execution facts and produces persistent, inspectable environmental monitoring records. The Frontend and Workflow Engine Adapter Layer are repository-owned parts of the wider prototype but are shown outside this logical boundary to distinguish presentation, workflow engine integration, and environmental monitoring responsibilities. Camunda 8, RabbitMQ, and the operational system provide the surrounding execution and communication environment.

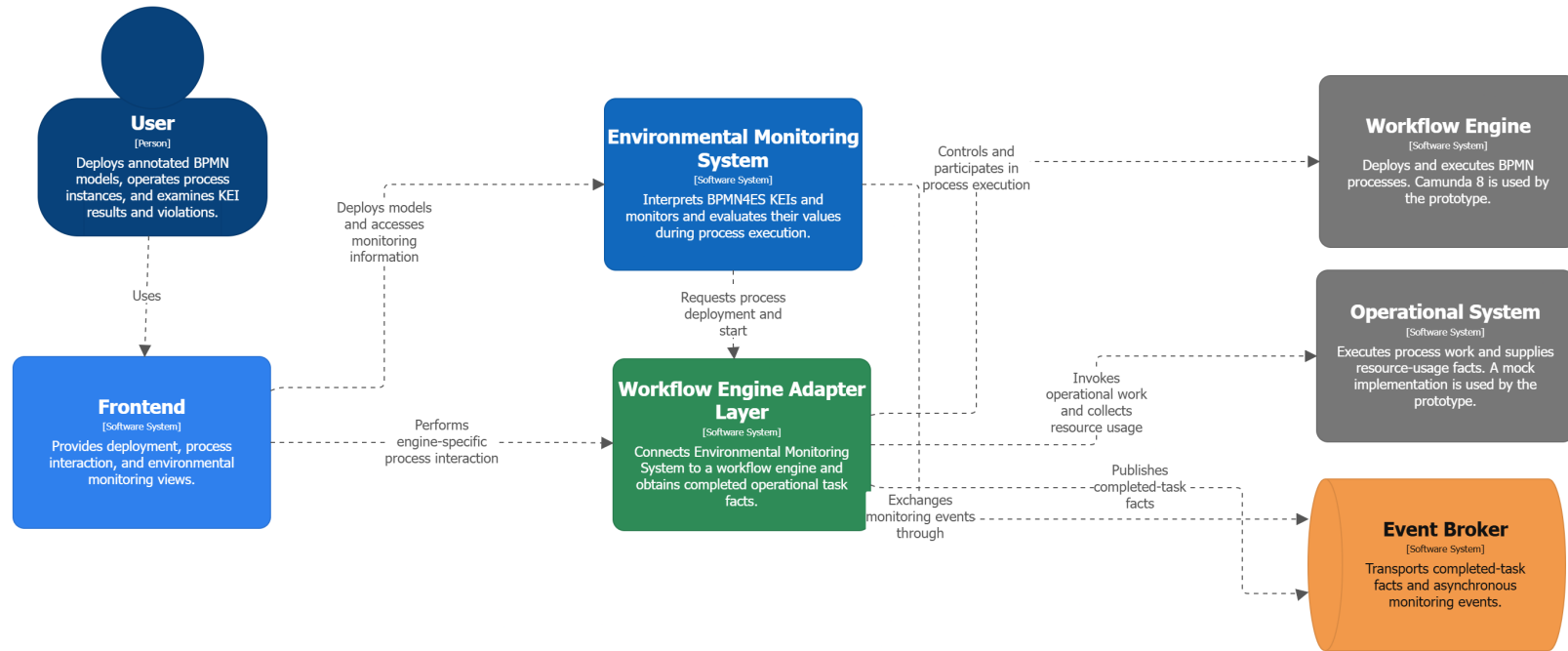


Figure 4.1: System context of the Environmental Monitoring System

The resulting structure follows the principal separation proposed by the reference architecture of Medema et al. [5]. Camunda 8 and the Workflow Engine Adapter Layer occupy the process-enactment side, while the Environmental Monitoring System provides the separate monitoring side. RabbitMQ transports execution facts and subsequent monitoring events between the participating components. The prototype further divides the monitoring responsibility into model registration, execution observation, environmental calculation, target evaluation, and result persistence.

The reference architecture presents calculator services as external collaborators of the environmental monitoring component rather than as parts of that component. The prototype retains calculation as a separately deployable responsibility but places the implemented CO2 Calculation service within the Environmental Monitoring System, alongside the services responsible for model registration, observation, evaluation, and result management. This represents the ownership and deployment structure of the realised prototype; it does not imply that calculator services must generally be internal to an environmental monitoring system. The depicted Frontend provides the operator-facing entry point to the wider prototype. The following design decisions explain how these boundaries and responsibilities were established.

4.3 Architectural Design Decisions

Each architectural design decision (ADD) follows the same record structure: decision, options considered, rationale, and consequences. Decision status records whether the choice is accepted, while implementation status identifies where it is realised in the prototype. Neither status by itself establishes satisfaction of a functional or quality requirement.

4.3.1 ADD-001: Parse Model Annotations and Register an External Runtime Mapping

Decision status: Accepted.

Implementation status: realised.

Decision

The Process Registry parses the submitted annotated BPMN model before deployment and associates each supported BPMN4ES KEI annotation with its containing BPMN element. After the workflow engine returns the deployment and process-definition identifiers, the Process Registry registers the original model, the engine identifiers, the annotated elements, and their KEI identifiers, units, optional targets, and display metadata through Monitoring Results. The workflow engine executes the submitted model without interpreting the annotations as native execution concepts.

Options considered

- **Engine-native metadata:** Transform the annotations into engine-specific variables, input mappings, or extensions interpreted by the workflow engine. This would place the metadata close to execution but couple its representation to Camunda.
- **Repeated model parsing:** Retain the model and parse it again whenever metadata is required. This would avoid a separate registered representation but repeat the interpretation and leave engine identifiers to be reconciled during every lookup.
- **External runtime mapping:** Parse the annotations during deployment and register them against the identifiers assigned by the engine. This preserves the modelling representation but introduces a separate consistency boundary.

Rationale

The external mapping preserves the annotated model as the source of the KEI definitions while respecting the black-box workflow engine constraint. The parser isolates BPMN4ES interpretation from deployment and downstream monitoring.

Consequences

- The parser becomes the transformation point between modelling-level annotations and their runtime representation.
- Runtime lookup depends on consistency between registered BPMN-element identifiers and the identifiers reported during execution.
- Parsing occurs before deployment, but registration occurs afterwards because it requires engine-assigned identifiers. Deployment and registration do not form one transaction; the engine deployment can therefore succeed before registration fails.

4.3.2 ADD-002: Isolate Engine Integration and Observe Configured Tasks Through a Worker

Decision status: Accepted.

Implementation status: Realised.

Decision

Camunda-specific deployment, instance, task, and job interactions remain within the Camunda Integration and Camunda Worker services. Service tasks configured for the prototype's worker job type are observed through a Camunda job worker. The worker performs the delegated operational action, receives the engine execution identifiers and resource-usage facts, and publishes them without exposing Camunda client objects to the monitoring services.

Options considered

- **Workflow-engine modification:** Extend the engine so that environmental monitoring becomes part of its native execution semantics. This would provide direct access to engine state but would violate the black-box constraint.
- **Engine life-cycle or history-backed observation:** Obtain activity events and their start and completion timestamps from engine life-cycle hooks or an exported execution-history source. This could provide authoritative activity timestamps and broaden element coverage, but would require an additional engine-specific integration mechanism and correlation with the operational facts. It would also not directly provide the resource-usage data produced during delegated operational work.
- **Worker-side observation:** Capture the execution context where the configured external service work is performed. This uses a supported Camunda boundary and provides both execution identifiers and operational facts, but limits observation to service tasks configured for the selected worker type.

Rationale

A Camunda job worker receives the process-definition, process-instance, element-instance, and BPMN-element identifiers required by the prototype [19], [20]. It also represents the boundary at which delegated operational work is performed and its resource-usage facts become available. Keeping this interaction within the adapter layer preserves the black-box engine constraint and prevents Camunda client types from entering the monitoring pipeline. Connectors, execution listeners, and history-backed observation provide alternative integration semantics but do not combine the selected delegated-work boundary with the required operational facts as directly [21], [22].

The mock operational service supplies deterministic resource-usage facts independently of the elapsed workflow execution time. Engine timestamps would therefore describe prototype execution latency rather than the duration of the simulated operational work and would not provide a suitable basis for the environmental calculation. History-based timestamp enrichment was consequently deferred.

Consequences

- The workflow engine remains unmodified, and engine-specific dependencies remain at identifiable integration boundaries.
- Observation is limited to service tasks configured for the selected worker type. The realised prototype does not observe arbitrary BPMN elements or obtain authoritative activity start and completion timestamps.
- Environmental calculations use the resource-usage facts supplied by the operational service rather than durations inferred from workflow engine timestamps.
- Replacing Camunda would require alternative command and execution adapters with equivalent identifiers and observation semantics. The architecture localises this change but does not demonstrate portability.
- The worker still depends synchronously on the operational service and on successful publication to the broker.

4.3.3 ADD-003: Decouple Downstream Monitoring Through Asynchronous Messaging

Decision status: Accepted.

Implementation status: Realised.

Decision

After operational work succeeds, the worker publishes an engine-task-completed event through a message broker before completing the corresponding engine job. Observation, calculation, evaluation, and result persistence communicate downstream through brokered events rather than through a synchronous request chain initiated by the worker. The prototype realises this broker with RabbitMQ. Synchronous communication is retained where an immediate response is required, including workflow engine commands and registered-annotation lookup.

Options considered

- **Synchronous monitoring chain:** The worker calls observation, calculation, evaluation, and persistence before completing the job. This simplifies ordering but exposes workflow progression to every downstream consumer's availability and latency.
- **Shared-database polling:** Downstream components discover completed work through a shared store. This avoids a broker but obscures ownership and introduces polling and schema coupling.
- **Brokered events:** The worker publishes completed-task facts and downstream stages communicate asynchronously. This removes direct downstream dependencies from the worker but introduces eventual consistency and messaging operations.

Rationale

Calculation and target evaluation are monitoring concerns and should not become synchronous prerequisites for workflow progression. Brokered events remove the availability and processing time of downstream consumers from the worker's request path once publication has succeeded.

Consequences

- The worker does not call or await Observation, CO2 Calculation, KEI Evaluation, or Monitoring Results.
- Monitoring records become eventually consistent with workflow execution.
- RabbitMQ remains a worker dependency because publication precedes engine-job completion.
- The architecture requires correlation information and compatible producer-consumer message representations.
- The prototype does not provide a transactional outbox, an end-to-end exactly-once guarantee, or a complete dead-letter and recovery strategy.

4.3.4 ADD-004: Enrich Execution Facts Before Calculation

Decision status: Accepted.

Implementation status: Realised.

Decision

The worker publishes an engine-task-completed event containing execution context and operational resource-usage facts. Observation resolves the registered KEI annotations for the reported process definition and BPMN element and transforms the combined information into one calculation request for each applicable distinct KEI. Calculators consume these requests without performing their own model lookup.

Options considered

- **Enrichment in the worker:** The worker retrieves model annotations and constructs indicator-specific requests. This removes a pipeline stage but couples workflow execution to the monitoring model.
- **Enrichment in every calculator:** Each calculator receives execution facts and independently retrieves model metadata. This keeps the worker smaller but duplicates lookup and mapping responsibilities across calculators.
- **Dedicated observation enrichment:** Observation joins execution facts with registered annotations before calculator routing. This adds one pipeline stage but prevents both the worker and calculators from owning metadata lookup.

Rationale

Dedicated enrichment keeps the worker unaware of modelled KEI semantics and keeps calculators independent of registered-model lookup. It makes the execution-to-calculation transformation explicit and independently diagnosable.

Consequences

- Calculation requests contain both execution correlation and the modelled KEI information needed by a calculator.
- Missing annotations and missing resource-usage data are detected before a calculation request is published.
- Observation depends synchronously on the registered-model lookup exposed by the Process Registry.

4.3.5 ADD-005: Separate Responsibilities into Deployable Services

Decision status: Accepted.

Implementation status: Realised.

Decision

Annotated-model deployment coordination, Camunda commands, service-task execution, observation enrichment, carbon-emission calculation, target evaluation, and monitoring-result persistence are assigned to separately deployable services. Each application database is accessed only through the service that owns it.

Options considered

- **Single application:** Implement the complete runtime pipeline in one deployable application. This reduces deployment overhead but places engine, environmental, and persistence dependencies in one release unit.
- **Modular monolith:** Retain one deployment unit while separating responsibilities through internal modules. This preserves code-level separation with fewer runtime contracts, but environmental and engine dependencies still share one release boundary.
- **Deployable services:** Assign the responsibilities to services communicating through synchronous interfaces and brokered events. This makes deployment and ownership boundaries explicit at the cost of distributed-system overhead.

Rationale

Model parsing depends on BPMN4ES, engine interaction depends on Camunda, environmental calculation depends on reference data and formulae, and monitoring queries depend on a persistent domain model. Separate services were selected to preserve these boundaries at deployment level and to make runtime dependencies and data ownership visible, while introducing the distributed-system costs listed below.

Consequences

- Monitoring Results owns registered process models and monitoring records, while CO2 Calculation owns its resource profiles and emission factors. Camunda remains authoritative for workflow state.
- Process Registry coordinates model registration but does not maintain a separate durable model store.
- Separately deployed services introduce configuration, deployment, observability, and contract-coordination overhead.
- Independent deployment does not imply that message contracts can evolve independently.

4.3.6 ADD-006: Route Calculation Requests by KEI Identifier

Decision status: Accepted.

Implementation status: Realised for the carbon emissions route.

Decision

Observation derives the calculation-request route from the KEI identifier. A calculator supports an indicator by binding its queue to the corresponding route, consuming the established calculation request contract, and publishing a compatible completed-calculation event.

Options considered

- **In-process strategy registry:** A central calculator selects an implementation through a strategy interface or dispatcher. This provides one entry point but places every strategy in the same application and release boundary.

- **Shared calculation queue:** All calculators consume one route and inspect each request to determine whether it is supported. This simplifies publication but permits an inappropriate consumer to receive a request.
- **Indicator-specific routing:** Each calculator binds to the route of the KEI it supports. This keeps selection in broker configuration but requires a route and binding for every supported indicator.

Rationale

Routing by KEI identifier keeps calculator selection outside the workflow worker, model parser, and registration representation. It provides a boundary at which a calculator with different logic and reference data could be introduced.

Consequences

- Architecturally, a calculator requires an inbound request consumer, indicator-specific logic and any required reference data, a result publisher, and a queue binding for its KEI route.
- An external calculation system can be connected through an adapter that consumes the internal request, sends the required inputs to the external interface, and includes the original KEI and execution fields when publishing the completed-calculation event.
- Adding a calculator that can use the established annotation and request fields does not require changes to the worker or to the process-model registration path and schema.
- Deployment configuration, a route binding, the calculator implementation, and suitable operational and reference data are still required.
- When RabbitMQ and the calculation-request exchange are available but no queue binding matches the request route, the request is returned to Observation's publisher and logged.
- Only the carbon-emissions calculator is implemented. The decision identifies an extension boundary but does not experimentally demonstrate calculation modifiability.

4.3.7 ADD-007: Represent Service Interactions Through Explicit Compatible Contracts

Decision status: Accepted.

Implementation status: Realised.

Decision

Each configured producer-consumer route has a defined domain message containing the correlation and domain data required by its consumers. Producers and consumers maintain compatible local types for that message.

Options considered

- **Informal payloads:** Exchange untyped maps or loosely documented fields. This reduces initial ceremony but defers structural errors to runtime processing.
- **Shared binary contract library:** Distribute common message classes to every producer and consumer. This removes duplicated types but couples services to releases of the shared library.
- **Service-local message representations:** Maintain explicit producer and consumer types whose compatibility is coordinated at each route. This preserves local ownership but duplicates representations.

Rationale

The contracts prevent types internal to services and the workflow worker from crossing boundaries and expose the correlation data required by the distributed pipeline. Local message types avoid a shared code dependency, while the message-compatibility requirement governs their structural agreement.

Consequences

- Each inbound adapter deserialises the message, while the receiving application processor validates it before continuing with domain processing.
- Producer and consumer representations are duplicated across services and must agree on field names, types, optionality, and enumeration values.
- An incompatible change requires coordinated changes and tests across all affected producers and consumers.
- The prototype has no shared schema registry or automated compatibility governance.

4.3.8 ADD-008: Persist Activity-Level Results and Derive Violations

Decision status: Accepted.

Implementation status: Realised.

Decision

Monitoring Results persists one activity-level monitoring record for each process instance, element instance, and KEI annotation when either a calculation without a target or a target evaluation completes. A detected violation is represented by a persisted monitoring record whose evaluation status is VIOLATED; operator-facing violation queries and live updates are derived from this status rather than from a separate violation aggregate.

Options considered

- **Persist every intermediate event:** Store separate observation, calculation, evaluation, failure, and violation records. This provides a more complete processing history but duplicates the operator-facing occurrence and requires further projection logic.
- **Dedicated violation projection:** Create a second persistent model with acknowledgement or resolution state. This supports a violation life cycle that the prototype does not otherwise define.
- **Single activity-level result:** Persist the terminal successful result for the monitored execution and derive violation views from its evaluation status. This keeps inspection focused but does not retain every failed or incomplete stage.

Rationale

A calculation and its subsequent evaluation describe the same monitored activity occurrence. A single result avoids presenting them as duplicate operator-facing records. Because the prototype provides no acknowledgment, assignment, remediation, or resolution behaviour, a separate violation life cycle is not justified.

Consequences

- A result retains the execution and KEI context, calculation metadata, resource contributions, and optional evaluation outcome.
- A calculation without a target is persisted directly; a target-bearing calculation is persisted only after evaluation completes.

- Violation queries return monitoring records classified as VIOLATED; the term *active* in the current interface does not denote an implemented active/resolved life cycle.
- Handled calculation failures and other incomplete processing paths do not form part of the persistent monitoring-result model.

4.4 Architecture Views

The following views refine the system context into deployable containers and their principal relationships. The component views then refine the internal structure of the Process Registry and Monitoring Results containers. The temporal ordering of deployment and runtime interactions is presented separately through sequence diagrams.

4.4.1 Container View

Figure 4.2 presents the containers of the Environmental Sustainability Monitoring System and its relationships with the surrounding prototype systems. The realised monitoring-system boundary contains five application services and two service-owned data stores. The Frontend, Camunda-specific integration services, RabbitMQ, Camunda 8, and the Mock Operational Service remain outside this boundary.

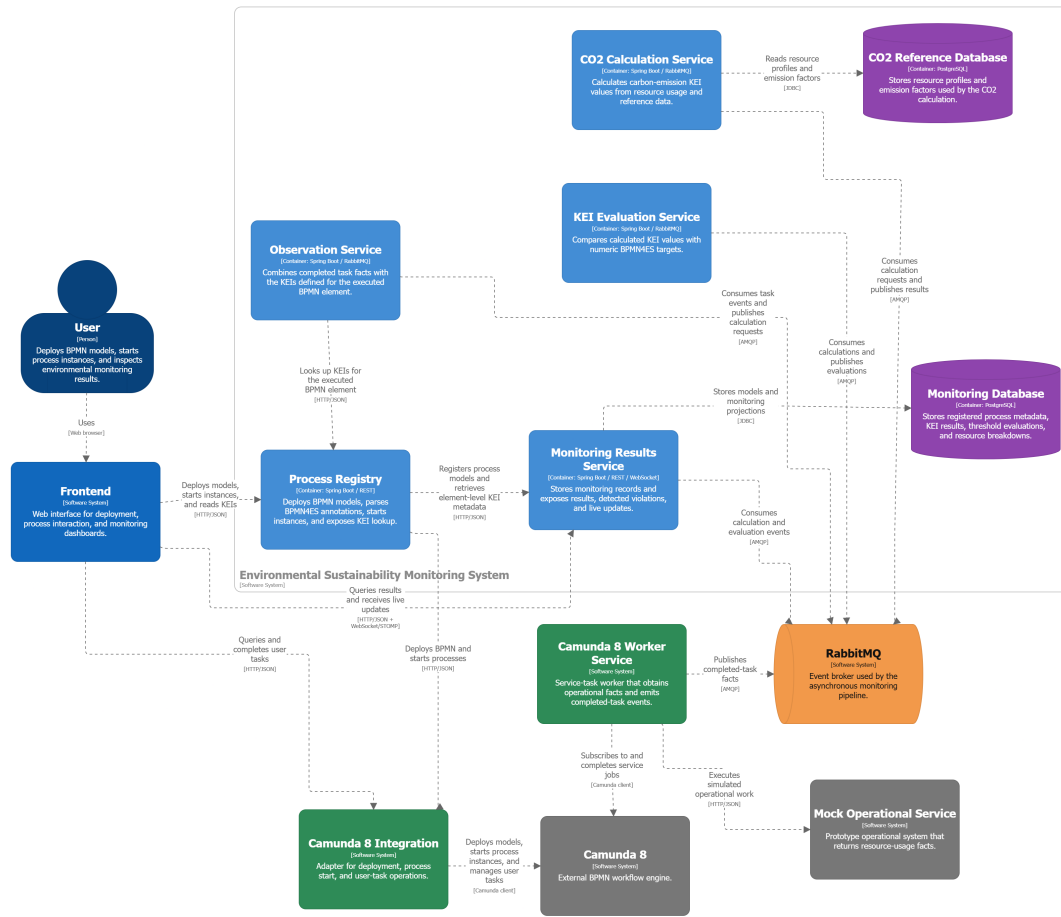


Figure 4.2: Containers and neighboring runtime systems of the Environmental Sustainability Monitoring System

The Process Registry coordinates annotated-model deployment and registration. It parses the submitted BPMN4ES model, delegates engine-specific operations to the Camunda 8 Integration service, and registers the deployed process definition and its element-level KEI metadata through Monitoring Results. It also provides the runtime lookup through which Observation retrieves the annotations associated with an executed BPMN element.

Observation combines the execution identifiers and resource-usage facts received from the Camunda 8 Worker Service with the registered modelling-level annotations. It publishes a calculation request for each distinct applicable KEI. RabbitMQ provides the asynchronous communication boundary between Observation, calculation, evaluation, and result persistence.

The CO2 Calculation Service implements the prototype’s supported **carbon-emissions** calculation using resource profiles and emission factors from the CO2 Reference Database. The KEI Evaluation Service compares calculated values with numeric targets when such targets are defined. Monitoring Results stores the registered model representation and activity-level monitoring records and exposes calculated values, evaluation outcomes, detected violations, and live updates to the Frontend.

The Camunda 8 Integration and Worker services isolate engine-specific communication. The Integration service handles deployment, instance, and user-task operations, while the Worker Service performs configured service-task work, obtains resource-usage facts from the Mock Operational Service, and publishes completed-task events. These services prevent the monitoring containers from depending directly on Camunda client objects.

The CO2 Reference Database is private to the CO2 Calculation Service, while the Monitoring Database is private to Monitoring Results. Other containers access these data through the interfaces of the owning service rather than through shared database schemas.

In the reference architecture, calculator services may operate as external collaborators of the environmental monitoring system. The realised prototype places the CO2 Calculation Service within the monitoring-system boundary because it is implemented and operated as part of the prototype, while retaining it as a separately deployable service behind a calculator-specific messaging interface.

4.4.2 Process Registry Component View

Process Registry forms the boundary between the submitted modelling representation, workflow engine deployment, and the persistent runtime mapping. Figure 4.3 separates its public Representational State Transfer (REST) API, application processors, BPMN4ES parser, and outbound adapters. The REST API accepts model deployment, process-instance start, process-level metadata lookup, and element-level annotation lookup requests and delegates each operation to the corresponding application processor.

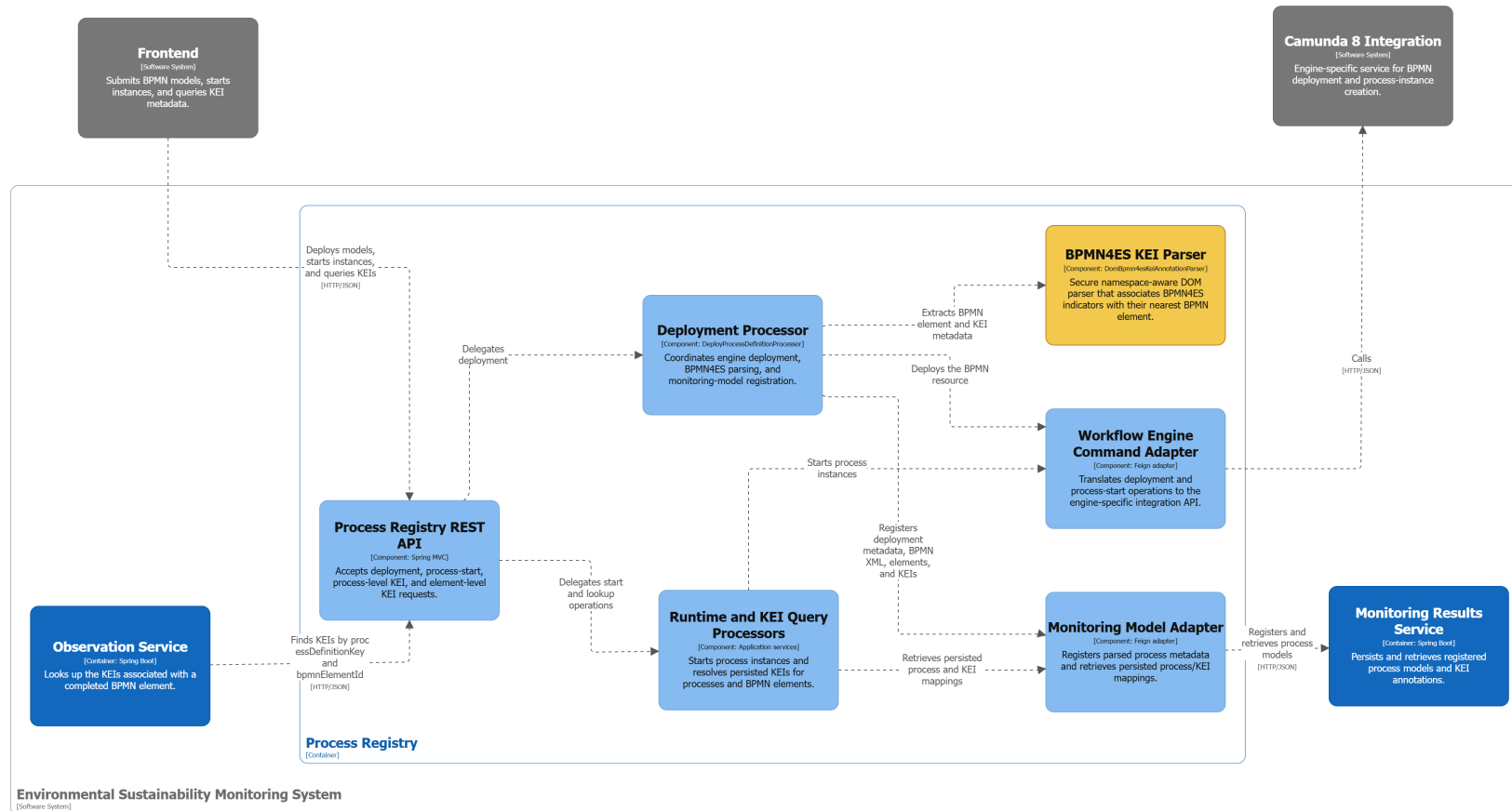


Figure 4.3: Process Registry components and external boundaries

The Deployment Processor coordinates the creation of the runtime mapping. Before invoking the workflow engine, it passes the submitted model to the BPMN4ES parser. The parser extracts the supported KEI annotations and associates them with their containing BPMN elements. The Workflow Engine Command Adapter then submits the model to Camunda 8 Integration and returns the identifiers assigned during deployment. Finally, the Monitoring Model Adapter supplies the original model, the extracted annotations, and the engine-assigned identifiers to Monitoring Results for registration.

The parser is separated from deployment coordination because it interprets modelling-specific XML rather than workflow engine state. It establishes the association between each annotation and its BPMN-element identifier before deployment, allowing the engine-assigned process-definition identity to be added afterwards. Workers, calculators, and result consumers therefore operate on the registered representation and do not need to interpret the original BPMN4ES document.

Process Registry owns no persistent registry store. Monitoring Results owns the durable process-definition, element, and KEI mappings, while Process Registry owns the workflow through which those mappings are created and retrieved. Runtime query processors use the Monitoring Model Adapter to obtain process-level or element-level annotations. Observation uses the element-level operation to resolve the annotations applicable to a reported execution. Process-instance creation is handled separately by delegating the request through the Workflow Engine Command Adapter.

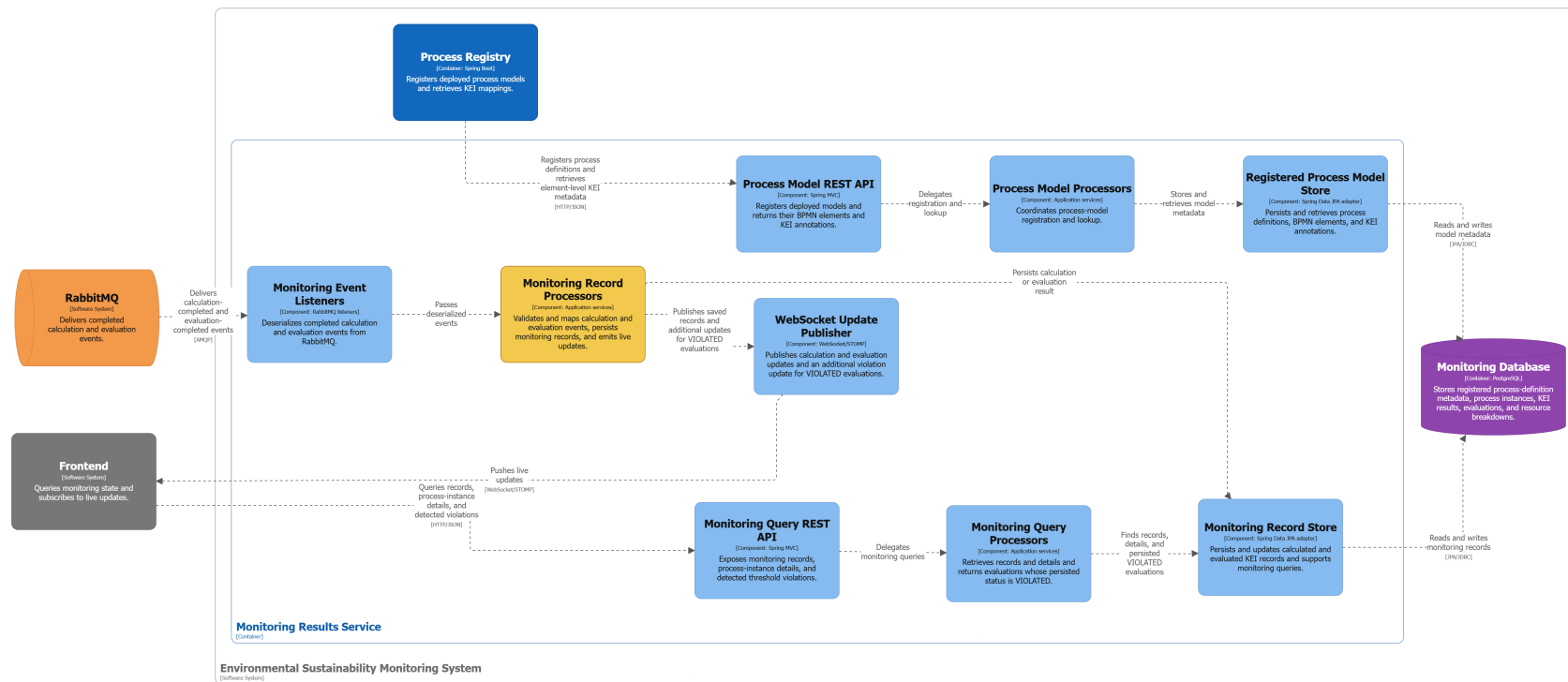
The application processors depend on parser and client interfaces rather than directly on Document Object Model (DOM) parsing, Camunda APIs, or the Monitoring Results transport representation. This keeps the coordination logic independent of the concrete mechanisms used at those boundaries and confines external-system translation to the corresponding adapters.

Deployment and registration do not form a single transaction. Parsing occurs before the engine call, but registration can only occur after the engine has returned its deployment identifiers. Consequently, an engine deployment may succeed even if subsequent registration fails, leaving the deployed definition unavailable to the monitoring path until registration is completed. Runtime annotation lookup also depends synchronously on the availability of the registered model representation.

The concrete REST operations, parser rules, XML security configuration, client representations, and error handling are described in Chapter 5.

4.4.3 Monitoring Results Component View

Monitoring Results owns the persistent representation of both registered process models and activity-level environmental results. Figure 4.4 shows two distinct input paths. Process Registry registers deployed models synchronously through the Process Model REST API, while RabbitMQ delivers completed calculation and evaluation events asynchronously. Query and WebSocket boundaries expose the resulting state to the Frontend.



Monitoring Results Service - C4 Component Diagram
 Friday, July 17, 2026 at 12:00 AM Central European Summer Time

Figure 4.4: Monitoring Results components, stores, and presentation boundaries

The Process Model Processors coordinate registration and retrieval through the Registered Process Model Store. The registered representation relates an engine-assigned process definition to its original BPMN model, annotated elements, and modelling-level KEI metadata. Monitoring Results, rather than Process Registry, owns this durable state. The representation subsequently provides the model context against which runtime results are persisted and queried.

Monitoring Event Listeners form the asynchronous input boundary. They deserialise completed calculation and evaluation events and pass them to the Monitoring Record Processors. These processors validate the incoming representation, map its execution, annotation, calculation, resource, and optional evaluation data into a monitoring record, and persist it through the Monitoring Record Store.

A calculation without a configured target produces a monitoring record when calculation completes. When a target is present, persistence is deferred until the corresponding evaluation event arrives. The evaluation event retains the completed calculation and adds the target comparison and resulting status. Calculation and evaluation are therefore represented as one activity-level record rather than as unrelated intermediate records.

Each monitoring record is associated with a registered process definition, process instance, element instance, BPMN element, and KEI annotation. This association preserves the modelling and execution context of the result and prevents the Monitoring Database from becoming an independent source of uncorrelated environmental values. It also means that the corresponding model, element, and annotation must already be registered before a monitoring event can be persisted successfully.

The Monitoring Query Processors retrieve monitoring records, process-instance details, and detected violations through the Monitoring Record Store. A violation is not stored as a separate aggregate. It is a monitoring record whose persisted evaluation status is `VIOLATED`.

The WebSocket Update Publisher sends calculation and evaluation notifications after the corresponding record has been persisted. A violated evaluation additionally produces a violation notification. These notifications support immediate presentation but do not constitute a second source of truth: a client that misses an update can retrieve the persisted state through the query interface.

The Registered Process Model Store and Monitoring Record Store are separate persistence boundaries but are backed by the same service-owned Monitoring Database. Other services access this state only through the APIs owned by Monitoring Results.

4.5 Deployment and Runtime Flows

The structural views are complemented by two dynamic views. The first shows how modelling-level data and engine-assigned identifiers become one registered model. The second shows where workflow execution ends and asynchronous monitoring begins.

4.5.1 Deployment Flow

Figure 4.5 shows how the modelling-level annotations and engine-assigned identifiers are combined into the runtime mapping. The flow begins when the user submits an annotated BPMN resource. Process Registry parses the model, delegates its deployment to the workflow engine integration boundary, and registers the resulting process definition together with its BPMN elements and KEI annotations in Monitoring Results.

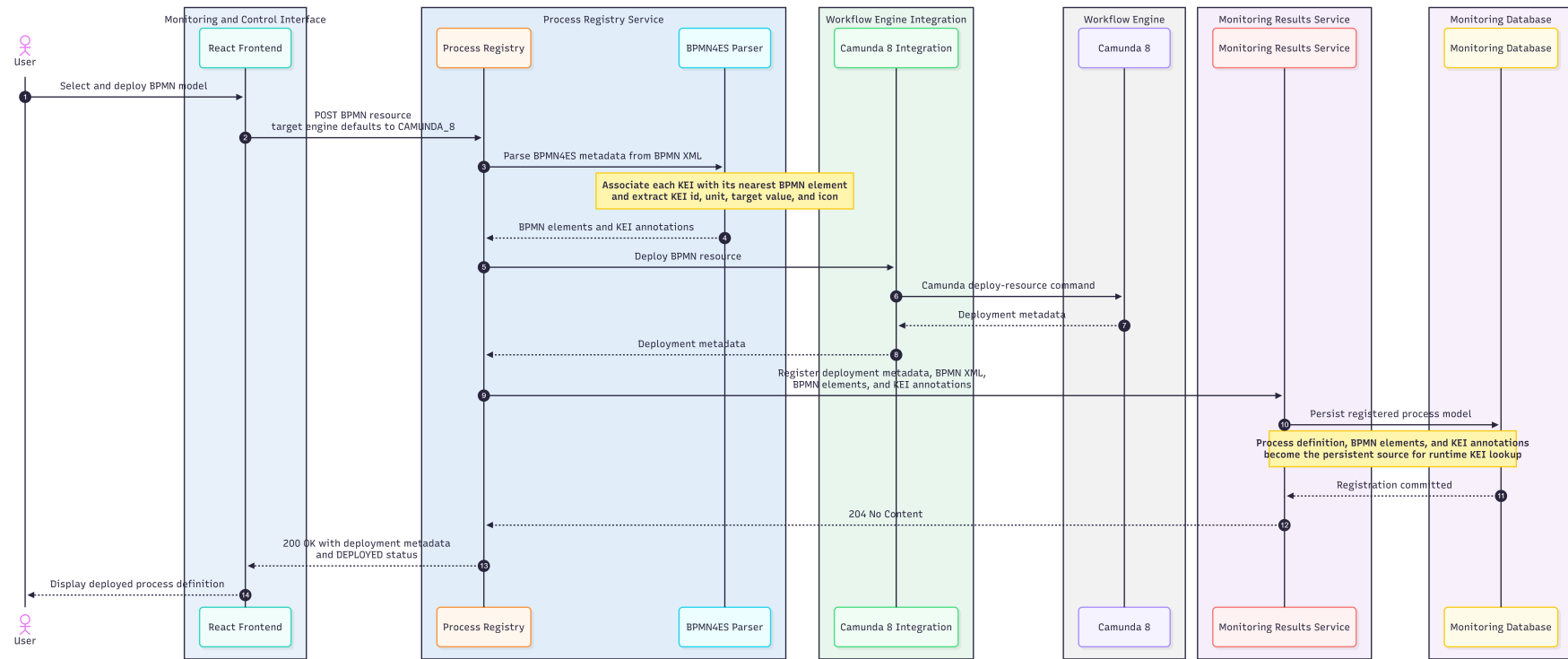


Figure 4.5: Deployment and registration sequence

Parsing precedes deployment so that the annotations are extracted from the submitted modelling representation before the engine is invoked. A model that cannot be parsed therefore fails before an engine deployment is attempted. Successful deployment supplies the engine-assigned identities needed to correlate later execution events with the registered model. Process Registry reports success to the Frontend only after Monitoring Results has accepted the combined representation.

Deployment and registration nevertheless cross two independently owned state boundaries and do not form one transaction. If registration fails after Camunda has accepted the model, the Process Registry request fails but the deployed process definition remains in the engine without a corresponding monitoring mapping. The realised prototype provides no distributed rollback or automatic reconciliation for this condition.

4.5.2 Runtime Monitoring Flow

Figure 4.6 presents the transition from synchronous service-task execution to the asynchronous monitoring pipeline. It begins after the annotated model has been deployed and registered and a process instance is already running. When Camunda activates a configured service-task job, the worker receives its execution identifiers and process variables, invokes the Mock Operational Service, and obtains the resource-usage facts produced for the delegated work.

The worker publishes an engine-task-completed event and then sends the job-completion command to Camunda. It does not call or await Observation, CO2 Calculation, KEI Evaluation, or Monitoring Results.

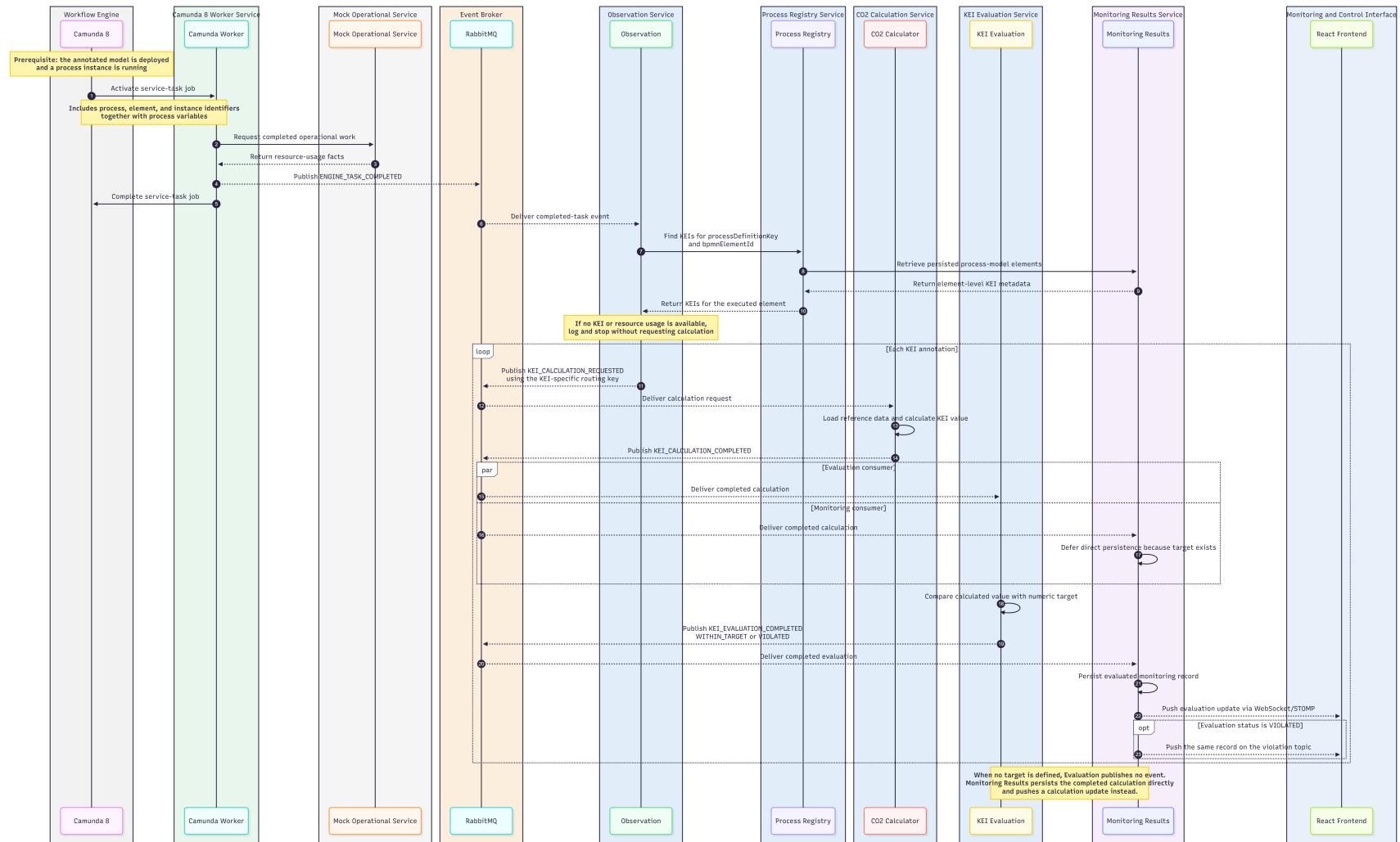


Figure 4.6: Runtime observation, calculation, evaluation, and monitoring-result sequence

The publication call precedes the engine job-completion command. RabbitMQ may, however, deliver the event before, during, or after that command because the worker does not control consumer scheduling. If publication fails synchronously, the worker does not send the completion command. No transaction spans event publication and engine-job completion, and successful publication does not establish an end-to-end delivery guarantee.

After receiving the event, Observation uses the process-definition and BPMN-element identifiers to retrieve the corresponding annotations through Process Registry. This is the final synchronous inter-service lookup in the monitoring path. Observation combines the returned metadata with the execution context and resource-usage facts and publishes one calculation request for each distinct applicable KEI. RabbitMQ uses the KEI-specific route to select a calculator; the realised route leads to the CO2 Calculation Service, which loads its reference data and publishes the calculated value.

The completed calculation is delivered independently to KEI Evaluation and Monitoring Results. If the annotation has no target, KEI Evaluation publishes no event and Monitoring Results persists the calculation directly. If a non-blank numeric target is present, Monitoring Results defers direct persistence while KEI Evaluation compares the calculated value with the maximum target. A value less than or equal to the target is classified as `WITHIN_TARGET`. A greater value is classified as `VIOLATED`. Monitoring Results then persists the calculation and evaluation as one activity-level record and publishes an evaluation update. A violated evaluation additionally produces an update on the violation channel.

The figure presents the successful processing routes. Missing annotations, missing resource usage, an unbound calculator route, a non-numeric target, or a handled calculation failure stops the affected monitoring step and produces diagnostic logging rather than a successful monitoring record. These downstream conditions do not reverse an already completed workflow job.

4.6 Communication Strategy

The architecture combines synchronous request-response communication with asynchronous event messaging. Synchronous communication is used where a component requires an immediate result before continuing. This includes model deployment, process-instance creation, user-task interaction, model registration, annotation lookup, and retrieval of operational resource-usage facts. REST is used between prototype services, while Camunda-specific commands and job interactions remain behind the engine adapter boundary.

The downstream monitoring path uses RabbitMQ. After completing the delegated operational work and successfully publishing the completed-task event, the worker completes the corresponding engine job without calling or awaiting Observation, CO2 Calculation, KEI Evaluation, or Monitoring Results. Subsequent hand-offs between observation, calculation, evaluation, and result persistence also use asynchronous messages, although Observation performs a synchronous annotation lookup when processing the completed-task event. Routing by KEI identifier selects the applicable calculator, while completed calculation events can be consumed independently by evaluation and result management.

Monitoring Results exposes persisted state through synchronous query interfaces and publishes live updates to connected Frontend clients through Simple Text Oriented Messaging Protocol (STOMP) over WebSocket. The live-update channel improves immediacy but is not a durable source of monitoring state. A client that disconnects or misses an update can retrieve the authoritative persisted records through the query interface.

Concrete endpoints, message representations, routing configuration, and WebSocket destinations are described in Chapter 5.

Chapter 5. Solution Realisation

This chapter describes how the architecture in Chapter 4 is realised in the prototype repository¹. The architectural views already establish the services, their responsibilities, and the principal deployment and runtime flows. The concern here is therefore the implementation structure behind those views: module boundaries, service-local contracts, model parsing, resource-usage data supplied by Mock Operational Service, carbon-emission calculation, messaging configuration, persistence, and presentation. The listings are extracted from the realised prototype.

5.1 Implementation Organisation and Deployment

The backend uses Java 21, Spring Boot 4.0.6, and Maven. Seven services use the same four-module structure, while the Camunda Worker is a single Maven application. The browser application is implemented separately with React, TypeScript, and Vite. The root `pom.xml` is a build aggregator for the eight backend services; it is not a shared application module.

Table 5.1: Repository units and implementation form

Repository unit	Implementation form	Included deployables
Layered Java services	Maven parent with api, application, infrastructure, and bootstrap modules	Process Registry, Camunda 8 Integration, Observation, Mock Operational, CO2 Calculation, KEI Evaluation, and Monitoring Results.
Camunda 8 Worker	Single Spring Boot Maven application	The worker that receives and completes Camunda jobs and publishes completed operational work.
Frontend	React and TypeScript application built with Vite	The browser interface for deployment, process interaction, monitoring records, violations, and live updates.

The common Maven structure follows a ports-and-adapters arrangement. Application processors depend on outbound interfaces, while infrastructure modules provide the Hypertext Transfer Protocol (HTTP), messaging, persistence, and parsing adapters that implement those interfaces. Table 5.2 records the purpose of each module and representative packages.

¹<https://github.com/SimeonStoyanov01/gbpm-runtime>

Table 5.2: Common backend module and package structure

Module	Contents	Representative packages
api	Operation interfaces and the models or event representations accepted and returned by those operations.	api.operations, api.model, api.events.
application	Processors implementing the operations, mapping or factory logic, and outbound ports required from external systems.	application.processors, application.mapper, application.factory, application.out.
infrastructure	Adapters for REST, HTTP clients, RabbitMQ, Java Persistence API (JPA), XML parsing, and WebSocket publication, according to the needs of the service.	infrastructure.rest, infrastructure.client, infrastructure.messaging, infrastructure.persistence, infrastructure.xml, infrastructure.websocket.
bootstrap	The executable Spring Boot application, external configuration, and resources used to assemble the other three modules.	The service application class, application.yml and service-owned initialisation resources.

The Frontend is organised by interaction boundary. Its **api** folder contains backend clients and shared response types, **components** contains reusable presentation elements, **features** contains the process, record, violation, dashboard, and live-event views, and **websocket** contains the STOMP client.

Each backend service has a multi-stage Dockerfile. Maven builds the executable artefact in a Java 21 build image, after which only the resulting Java archive (JAR) is copied into a Java 21 runtime image. Local deployment is divided into the three Compose projects shown in Table 5.3. All three attach to the externally created **runtime-local** network. RabbitMQ, Camunda, Elasticsearch, and both PostgreSQL databases use named volumes; the Frontend is started separately through Vite and is not Dockerised by the supplied deployment.

Table 5.3: Local container deployment

Compose project	Containers	Purpose
infra	RabbitMQ, CO2 Calculation PostgreSQL, and Monitoring Results PostgreSQL.	Runs the broker and the two service-owned databases. The CO2 database is initialised from the Structured Query Language (SQL) file that defines its reference-data tables and demonstration rows.
camunda8	Camunda orchestration, Elasticsearch, Camunda 8 Integration, and Camunda 8 Worker.	Provides the workflow runtime and the two engine-specific adapter services.

Compose project	Containers	Purpose
runtime	Process Registry, Mock Operational, Observation, CO2 Calculation, KEI Evaluation, and Monitoring Results.	Runs the services that implement model registration and the environmental monitoring path.
Outside Compose	Frontend.	Runs as a Vite application and connects to the backend interfaces exposed by the local deployment.

5.2 BPMN4ES Model Translation

Process Registry defines `Bpmn4esKeiAnnotationParser` as an application output port and implements it with `DomBpmn4esKeiAnnotationParser` in the infrastructure module. The deployment processor invokes this parser before it calls the workflow engine deployment port. It later combines the parsed BPMN4ES representation with the identifiers returned by the engine and sends the result through the monitoring-model registration port.

Listing 5.1 presents a simplified representation of one annotation in the submitted model. The namespace declaration binds the `bpmn4es` prefix to the BPMN4ES namespace Uniform Resource Identifier (URI) used by the parser.

```
<bpmn:definitions
  xmlns:bpmn="http://www.omg.org/spec/BPMN/20100524/MODEL"
  xmlns:bpmn4es="https://github.com/michel-medema/BPMN4ES">

  <bpmn:process id="Process_0xxgmzy">
    <bpmn:serviceTask
      id="Activity_1d0ugzo"
      name="Assemble Parts">
      <bpmn:extensionElements>
        <bpmn4es:environmentalIndicators>
          <bpmn4es:keyEnvironmentalIndicator
            id="carbon-emissions"
            unit="kg"
            targetValue="15"
            icon="co2"/>
          </bpmn4es:environmentalIndicators>
        </bpmn:extensionElements>
      </bpmn:serviceTask>
    </bpmn:process>
  </bpmn:definitions>
```

Listing 5.1: BPMN4ES annotation in the submitted process model

The parser searches the BPMN4ES namespace for `environmentalIndicators` containers. For each container, it walks upwards to the nearest BPMN model element that has an identifier and is not a diagram element. It then associates the element identifier, name, and type with every direct `keyEnvironmentalIndicator` child that has a non-blank `id`. The optional `unit`, `targetValue`, and `icon` attributes are retained. Listing 5.2 shows the element-association logic.

```
private Optional<ElementKeiAnnotations> findElementKeiAnnotation(
  Element environmentalIndicatorElement
) {
  Optional<Element> bpmnParentElement =
    findNearestBpmnParentWithId(environmentalIndicatorElement);
  if (bpmnParentElement.isEmpty()) {
    return Optional.empty();
  }
}
```

```

List<KeiAnnotation> keiAnnotations =
    findKeiAnnotations(environmentalIndicatorElement);
if (keiAnnotations.isEmpty()) {
    return Optional.empty();
}

return Optional.of(ElementKeiAnnotations.builder()
    .bpmnElementId(bpmnParentElement.get().getAttribute("id"))
    .elementName(findAttributeValue(
        bpmnParentElement.get(), "name").orElse(null))
    .elementType(bpmnParentElement.get().getLocalName())
    .keiAnnotations(keiAnnotations)
    .build());
}

private Optional<Element> findNearestBpmnParentWithId(Node node) {
    Node currentNode = node.getParentNode();
    while (currentNode != null) {
        if (currentNode instanceof Element currentElement
            && isBpmnElement(currentElement)
            && !isDiagramElement(currentElement)
            && !currentElement.getAttribute("id").isBlank()) {
            return Optional.of(currentElement);
        }
        currentNode = currentNode.getParentNode();
    }
    return Optional.empty();
}

```

Listing 5.2: Association of BPMN4ES annotations with their containing BPMN element

The parser is namespace-aware so that BPMN4ES extensions can be distinguished from BPMN and diagram elements. It recognises the namespace, element names, and attributes described above rather than performing full schema validation. A parsing failure is reported as an invalid process definition before the model is sent to Camunda.

After parsing, the deployment processor sends the unchanged model to Camunda and combines the returned deployment information with the extracted annotations. Listing 5.3 shows the complete coordination order. The registration call supplies Monitoring Results with the original model, the engine-assigned process definition and deployment identifiers, and the element-level annotations. The resulting representation makes each registered KEI retrievable by the process-definition and BPMN-element identifiers later reported during execution.

```

public DeployProcessDefinitionResponse process(
    DeployProcessDefinitionRequest request
) {
    List<ElementKeiAnnotations> elementKeiAnnotations =
        bpmn4esKeiAnnotationParser.parse(request.getBpmnXml());

    DeployProcessDefinitionResponse deployProcessDefinitionResponse =
        workflowEngineCommandClient.deployProcess(request);

    monitoringProcessModelClient.registerProcessModel(
        deployProcessDefinitionResponse,
        request.getBpmnXml(),
        elementKeiAnnotations
    );

    return deployProcessDefinitionResponse;
}

```

Listing 5.3: Coordination of model parsing, deployment, and registration

5.3 Worker Integration and Mock Operational Service

The Camunda Worker receives the engine identifiers and process variables for a job of type `default-worker`. It sends the BPMN element identifier and the configured `workObject` to Mock Operational Service. In a production integration this call would cross the boundary to the operational source of resource-use facts. In the prototype it instead performs a deterministic lookup in `mock-execution-runs.json`.

No formal JSON Schema is present. The structure of the entries in `mock-execution-runs.json` is defined by the `MockExecutionRunRecord` class shown in Listing 5.4. A record is selected by the combination of BPMN element identifier, work-object type, and material. The order identifier and work-object identifier are carried by the request but do not take part in this lookup.

```
// MockExecutionRunRecord
public class MockExecutionRunRecord {
    private String bpmnElementId;
    private String objectType;
    private String material;
    private String resourceName;
    private Double timeUsed;
    private String unit;
}

public class JsonExecutionRunLookup {
    private List<MockExecutionRunRecord> records;

    public Optional<MockExecutionRunRecord> findExecutionRun(
        String bpmnElementId,
        String objectType,
        String material
    ) {
        return records.stream()
            .filter(record ->
                record.getBpmnElementId().equals(bpmnElementId))
            .filter(record ->
                record.getObjectType().equals(objectType))
            .filter(record ->
                record.getMaterial().equals(material))
            .findFirst();
    }
}
```

Listing 5.4: Effective schema and selection rule of a mock execution record

Listing 5.5 contains the two configured records for the *Assemble Parts* activity. The lookup returns one resource-usage item constructed from the selected record. The record for a car supplies three hours of *Welder* use, whereas the record for a motorbike supplies one and a half hours.

```
[
  {
    "bpmnElementId": "Activity_1d0ugzo",
    "objectType": "car",
    "material": "steel",
    "resourceName": "Welder",
    "timeUsed": 3.0,
    "unit": "hour"
  },
  {
    "bpmnElementId": "Activity_1d0ugzo",
    "objectType": "motorbike",
    "material": "steel",
    "resourceName": "Welder",
    "timeUsed": 1.5,
    "unit": "hour"
  }
]
```

```

    "timeUsed": 1.5,
    "unit": "hour"
  }
]

```

Listing 5.5: Representative deterministic operational records

The mock service does not implement Popescu’s calculator. It supplies the runtime activity-data part of the calculation—resource name, usage value, and unit—using the automobile example and a service-task identifier. The file-based selection by BPMN element, object type, and material is specific to this prototype. Environmental characteristics of the resource are resolved later by CO2 Calculation rather than being returned by the operational source.

5.4 Observation Enrichment

Observation is the point at which the modelling-level annotations registered during deployment re-enter the runtime path. After validating an engine-task-completed event, it uses the reported process-definition and BPMN-element identifiers to retrieve the corresponding annotations through Process Registry. An empty lookup is logged and ends processing for that event.

When annotations are present, Observation checks that the completed operational work supplied at least one resource-use entry. It then combines the execution context, work-object type, resource usage, and one registered annotation into a calculation request. This produces one request for each annotation returned for the element, while keeping the worker independent of KEI metadata and the calculators independent of model lookup. Listing 5.6 shows this transformation.

```

private List<KeiAnnotation> findKeiAnnotations(
    EngineTaskCompletedEvent event
) {
    return keiAnnotationLookupClient.findKeiAnnotations(
        event.getExecution().getProcessDefinitionKey(),
        event.getExecution().getBpmnElementId()
    );
}

private void requestCalculations(
    EngineTaskCompletedEvent event,
    List<KeiAnnotation> keiAnnotations
) {
    List<ResourceUsage> resourceUsages = event.getResourceUsages();
    if (resourceUsages == null || resourceUsages.isEmpty()) {
        log.info(
            "Skipping KEI calculation request because resource usage data is missing: bpmnElementId={}",
            event.getExecution().getBpmnElementId()
        );
        return;
    }

    for (KeiAnnotation keiAnnotation : keiAnnotations) {
        KeiCalculationRequestedEvent calculationEvent =
            keiCalculationRequestedEventFactory.create(
                event.getExecution(),
                keiAnnotation,
                event.getWorkObjectType(),
                resourceUsages
            );

        keiCalculationRequestedEventPublisher.publish(calculationEvent);
    }
}

```

```

        log.info(
            "Published KEI calculation request event: keiId={},
resourceUsageCount={}",
            keiAnnotation.getId(),
            resourceUsages.size()
        );
    }
}

```

Listing 5.6: Enrichment of completed-task facts into KEI calculation requests

5.5 Service-Local Contracts and Event Routing

The services do not share a Maven contract library. Instead, each producer and consumer owns a local Java representation of the JSON message it sends or receives. This avoids a common compiled dependency but transfers coupling to the serialised contract. Table 5.4 shows where these representations are repeated. Consumer copies may add local validation annotations, but their field names and nested structures must remain compatible with the corresponding producer.

Table 5.4: Service-local message representations

Message representation	Producer	Consumer
EngineTaskCompletedEvent	Camunda 8 Worker	Observation.
KeiCalculationRequestedEvent	Observation	CO2 Calculation.
KeiCalculationCompletedEvent	CO2 Calculation	KEI Evaluation and Monitoring Results, each with a separate local copy.
KeiEvaluationCompletedEvent	KEI Evaluation	Monitoring Results.
KeiCalculationFailedEvent	CO2 Calculation	No consumer is configured in the local deployment.

The calculation request preserves the engine execution context and resource usages from the worker event and adds the selected KEI annotation. A completed calculation adds its event identity and time, calculator metadata, result, and resource breakdown. The evaluation event then contains the completed-calculation representation together with the target, difference, and classification. Replication therefore preserves the correlation fields as the message is extended at each processing boundary.

The concrete broker configuration uses durable topic exchanges and durable service-specific queues. Exact default destinations are listed in Table 5.5. The two queues bound to the completed-calculation route allow Evaluation and Monitoring Results to receive the same event independently.

Table 5.5: Configured RabbitMQ exchanges, routing keys, and queues

Exchange	Routing key	Bound queue
runtime.camunda8.events	engine.task.completed	runtime.observation. engine-task-completed.

Exchange	Routing key	Bound queue
runtime. observation.events	kei.calculation. requested.<keiId>	An indicator-specific calculator queue. The realised carbon-emissions binding uses runtime.co2-calculation.kei-calculation-requested.
runtime. calculation.events	kei.calculation. completed	runtime.evaluation. kei-calculation-completed and runtime.monitoring. kei-calculation-completed.
runtime. calculation.events	kei.calculation. failed	No queue is bound in the supplied deployment.
runtime. evaluation.events	kei.evaluation. completed	runtime.monitoring. kei-evaluation-completed.

Observation derives the calculator route rather than selecting a calculator in application code. Listing 5.7 shows that the configured prefix is extended with the KEI identifier. Mandatory publisher returns are enabled in the same publisher so that an unbound route is logged by Observation.

```
private String routingKeyFor(KeiCalculationRequestedEvent event) {
    return rabbitMqProperties.getCalculationRequestedRoutingKeyPrefix()
        + "." + event.getKei().getId();
}
```

Listing 5.7: Derivation of an indicator-specific calculation route

The CO2 service supplies the matching routing key and queue name in its external configuration, as shown in Listing 5.8.

```
runtime:
  messaging:
    calculation-request:
      exchange-name: ${RUNTIME_CALCULATION_REQUEST_EXCHANGE:runtime.observation.
events}
      routing-key: ${RUNTIME_CALCULATION_REQUEST_ROUTING_KEY:kei.calculation.
requested.carbon-emissions}
      queue-name: ${RUNTIME_CALCULATION_REQUEST_QUEUE:runtime.co2-calculation.kei-
calculation-requested}
```

Listing 5.8: Configured carbon-emissions calculation route

Spring uses these properties to bind the calculator's durable queue to the topic exchange. The listener consumes that queue and passes each successfully deserialised representation to the calculation operation, where Bean Validation checks its contents. Listing 5.9 shows both points at which the configured route becomes executable.

```
// CalculationRequestRabbitMqConfiguration
@Bean
public Binding calculationRequestBinding(
    Queue calculationRequestQueue,
    TopicExchange calculationRequestExchange,
    CalculationRequestRabbitMqProperties properties
) {
    return BindingBuilder
        .bind(calculationRequestQueue)
        .to(calculationRequestExchange)
        .with(properties.getRoutingKey());
}
```

```
// KeiCalculationRequestedEventListener
@RabbitListener(
    queues = "${runtime.messaging.calculation-request.queue-name}"
)
public void handle(byte[] payload) {
    KeiCalculationRequestedEvent event;
    try {
        event = readEvent(payload);
    } catch (IllegalArgumentException exception) {
        log.warn(
            "Skipping invalid KEI calculation request event: {}",
            exception.getMessage()
        );
        return;
    }

    calculateCo2operation.process(event);
}
```

Listing 5.9: CO2 calculation-request queue binding and listener

Adding a distinct calculator requires its implementation and a queue binding for the route formed from its KEI identifier. The Worker and Observation remain unchanged.

5.6 Carbon-Emission Calculator

CO2 Calculation obtains the runtime resource-use entries from the calculation request and owns the environmental reference data used to interpret them. Each entry names a resource and states how long it was used through its `timeUsed` value and unit. The calculator looks up a resource profile that specifies the resource’s fuel type and consumption rate, including the fuel unit and the unit of resource use to which the rate applies. It then looks up the emission factor configured for that fuel type. Both records are selected from the reference set named in the calculator configuration.

The service stores this reference data in two PostgreSQL tables. For the values used in this thesis, the *Welder* profile specifies five litres of diesel per hour, so its runtime usage must be reported in hours and the resulting fuel consumption is expressed in litres. The diesel record supplies an emission factor of 2.70553 kilograms per litre. Listing 5.10 shows the schema and the rows required for this example.

```
CREATE TABLE IF NOT EXISTS resource_profiles (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    reference_set_id VARCHAR(255) NOT NULL,
    name VARCHAR(255) NOT NULL,
    type VARCHAR(255) NOT NULL,
    fuel_per_use NUMERIC(19, 8) NOT NULL,
    fuel_type VARCHAR(255) NOT NULL,
    fuel_unit VARCHAR(255) NOT NULL,
    time_unit VARCHAR(255) NOT NULL,
    CONSTRAINT uk_resource_profiles_reference_set_name
        UNIQUE (reference_set_id, name)
);

CREATE TABLE IF NOT EXISTS emission_factors (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    reference_set_id VARCHAR(255) NOT NULL,
    fuel_type VARCHAR(255) NOT NULL,
    unit VARCHAR(255) NOT NULL,
    factor NUMERIC(19, 8) NOT NULL,
    CONSTRAINT uk_emission_factors_reference_set_fuel_type
        UNIQUE (reference_set_id, fuel_type)
);
```

```

);

INSERT INTO resource_profiles (
    reference_set_id, name, type, fuel_per_use,
    fuel_type, fuel_unit, time_unit
) VALUES (
    'popescu-demo-v1', 'Welder', 'atomic', 5.0,
    'Diesel', 'l', 'hour'
)
ON CONFLICT (reference_set_id, name) DO NOTHING;

INSERT INTO emission_factors (
    reference_set_id, fuel_type, unit, factor
) VALUES (
    'popescu-demo-v1', 'Diesel', 'l', 2.70553
)
ON CONFLICT (reference_set_id, fuel_type) DO NOTHING;

```

Listing 5.10: CO2 reference-data schema and representative seed rows

For each resource-use entry, the calculator requires the observed unit to match the profile's usage unit and the profile's fuel unit to match the factor unit. It computes the consumed fuel from the observed duration and profile rate, then applies the fuel-specific factor. Each resource contribution and the final sum are rounded half-up to four decimal places. Listing 5.11 contains the corresponding implementation.

```

private static final String RESULT_UNIT = "kg";
private static final int RESULT_SCALE = 4;

private CalculationOutcome calculate(
    KeiCalculationRequestedEvent request,
    CalculationMetadata calculation
) {
    requireSupportedKei(request);

    BigDecimal total = BigDecimal.ZERO;
    List<ResourceBreakdown> breakdown = new ArrayList<>();

    for (ResourceUsage resourceUsage :
        request.getInputs().getResourceUsages()) {
        ResourceBreakdown resourceBreakdown = calculateResource(
            resourceUsage, calculation.getReferenceSetId());
        total = total.add(resourceBreakdown.getEmissionValue());
        breakdown.add(resourceBreakdown);
    }

    return CalculationOutcome.succeeded(
        total.setScale(RESULT_SCALE, RoundingMode.HALF_UP),
        RESULT_UNIT,
        breakdown
    );
}

private ResourceBreakdown calculateResource(
    ResourceUsage resourceUsage,
    String referenceSetId
) {
    ResourceProfile resourceProfile = resourceProfileLookup
        .findResourceProfile(
            referenceSetId, resourceUsage.getResourceName())
        .orElseThrow(() ->
            CalculationFailureException.resourceProfileNotFound(
                referenceSetId,
                resourceUsage.getResourceName())
        );
}

```

```

        ));

    requireResourceUsageUnit(resourceUsage, resourceProfile);

    EmissionFactor emissionFactor = emissionFactorLookup
        .findEmissionFactor(
            referenceSetId, resourceProfile.getFuelType())
        .orElseThrow(() ->
            CalculationFailureException.emissionFactorNotFound(
                referenceSetId,
                resourceProfile.getFuelType()
            ));

    requireFuelUnit(resourceProfile, emissionFactor);

    BigDecimal timeUsed =
        BigDecimal.valueOf(resourceUsage.getTimeUsed());
    BigDecimal consumedAmount =
        timeUsed.multiply(resourceProfile.getFuelPerUse());
    BigDecimal emissionValue = consumedAmount
        .multiply(emissionFactor.getFactor())
        .setScale(RESULT_SCALE, RoundingMode.HALF_UP);

    return ResourceBreakdown.builder()
        .resourceName(resourceUsage.getResourceName())
        .usageValue(timeUsed)
        .usageUnit(resourceUsage.getUnit())
        .emissionValue(emissionValue)
        .unit(RESULT_UNIT)
        .build();
}

```

Listing 5.11: Resource-level carbon-emission calculation

This sequence implements the resource-based calculation structure described by Popescu: task resource usage is interpreted through a resource-consumption profile and a fuel-specific emission factor [9]. The calculator configuration names the demonstration reference set `popescu-demo-v1`. For the car entry in Listing 5.5, three hours of Welder use for a specific task consume fifteen litres of diesel and produce 40.5830 kilograms after rounding. A request containing several resource-use entries is handled in the same way for each entry, after which their contributions are added to the task result.

5.7 Target Evaluation, Persistence, and Presentation

KEI Evaluation parses a non-blank annotation target as a decimal maximum. It assigns `WITHIN_TARGET` when the calculated value is less than or equal to the target and `VIOLATED` when it is greater. Monitoring Results stores a completed calculation directly when no target is present. When a valid numeric target is present, it stores the later evaluation event instead.

The calculation and its optional evaluation are stored as one activity-level monitoring record. The persisted result relates the process definition, process instance, element instance, BPMN element, and KEI annotation to the calculated value, calculator metadata, resource breakdown, and optional target assessment. For an evaluated result, the target value, difference, evaluation time, and either `WITHIN_TARGET` or `VIOLATED` status are stored on that record. A violation is therefore retrieved by filtering monitoring records whose stored evaluation status is `VIOLATED`; it is not maintained as a separate persistent object.

Monitoring Results exposes the persisted projection through REST and publishes a saved record to connected Frontend clients through STOMP over a native WebSocket. Listing 5.12 shows the server-side configuration. Spring’s in-memory simple broker handles destinations

under `/topic`, and clients connect through `/ws/monitoring`.

```
@Configuration
@EnableWebSocketMessageBroker
public class MonitoringWebSocketConfiguration
    implements WebSocketMessageBrokerConfigurer {

    @Override
    public void configureMessageBroker(MessageBrokerRegistry registry) {
        registry.enableSimpleBroker("/topic");
        registry.setApplicationDestinationPrefixes("/app");
    }

    @Override
    public void registerStompEndpoints(StompEndpointRegistry registry) {
        registry.addEndpoint("/ws/monitoring")
            .setAllowedOriginPatterns("*");
    }
}
```

Listing 5.12: STOMP-over-WebSocket configuration

The publisher sends the same persisted monitoring-record representation to the topics in Table 5.6. The violation topic is an additional notification for an evaluated record.

Table 5.6: Frontend live-update topics

STOMP destination	Published record
<code>/topic/monitoring/calculations</code>	A persisted calculation for an annotation without a target.
<code>/topic/monitoring/evaluations</code>	A persisted result after numeric target evaluation.
<code>/topic/monitoring/violations</code>	The same persisted evaluated result when its status is VIOLATED.

The Frontend retrieves persisted results and violations through REST and subscribes to the three topics using `@stomp/stompjs`. Live notifications update the dashboard while the client is connected. They are not durable and do not replace the REST-backed state. The interface also uses `bpmn-js` to render the stored process model and place persisted results on their corresponding BPMN elements.

5.7.1 User Task Interaction

The Frontend includes a limited user task control so that the process can be completed without moving between the prototype and Camunda’s own interface. After starting an instance, it uses the REST interface exposed directly by Camunda Integration to query user tasks in the CREATED state and to complete them. The implemented view tracks only the process instance most recently started through that browser session. It does not allow managing user tasks for concurrent instances.

For a user task whose sole outgoing flow leads to an exclusive gateway with a default branch, Camunda Integration extracts the name of the boolean decision variable when exactly one other branch uses a condition of the form `= variable = true`. Examples in the automobile model include `designApproved`, `qualityCheckPassed`, and `finalInspectionPassed`. The interface then presents *Yes/No* and submits the selected Boolean value under that name. An ordinary user task is presented with a *Complete* action.

This control is provided for prototype convenience. Camunda can execute multiple process instances concurrently, and their user tasks can be completed through another client. Subsequent service task execution and monitoring are unaffected by the client used for completion. The Frontend therefore trades multi-instance task management for a self-contained demonstration path without coupling user-task interaction to the environmental monitoring pipeline.

5.7.2 Prototype Interface Evidence

The interface captures show the model and the operator-facing path through deployment, process interaction, persisted monitoring records, and live notifications.

Figure 5.1 shows the annotated automobile assembly model submitted to the prototype.

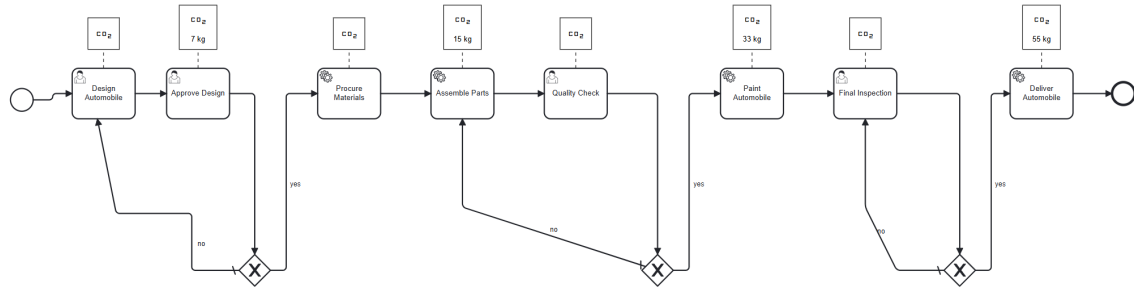


Figure 5.1: Annotated automobile assembly process used by the prototype

Figure 5.2 shows the controls used to submit the annotated model, start a registered definition with the demonstration variables, and complete an active user task. The deployment result exposes the identifiers required for registration and process start. The start result exposes the created instance key, after which active tasks are obtained through Camunda Integration.

Processes

Deploy BPMN

POST :8080 /api/process-definitions/deploy

BPMN FILE

Choose File

automobile_design_kei_notarget.bpmn

Deploy BPMN file

Deployed process

DEPLOYED

PROCESS	VERSION
Process_0xxgmzy	6
DEFINITION KEY	DEPLOYMENT KEY
2251799813693524	2251799813712969
BPMN RESOURCE	
automobile_design_kei_notarget.bpmn	

View KEIs

Use key for start form

Start Process Instance

POST :8080 /api/process-definitions/{key}/instances

PROCESS DEFINITION KEY

2251799813693524

VARIABLES JSON

```

{
  "orderId": "ORD-2041",
  "workObject": {
    "objectId": "OBJ-001",
    "type": "car",
    "material": "steel"
  }
}

```

Example payload for testing. Edit it to match the deployed process.

Start process instance

Started process instance

STARTED

PROCESS	VERSION
Process_0xxgmzy	6
DEFINITION KEY	INSTANCE KEY
2251799813693524	2251799813712972

Active User Tasks

GET :8090 /api/v1/camunda8/process-instances/{key}/user-tasks

No active user tasks for this instance.

(a) Annotated-model deployment

(b) Process-instance creation

Start Process Instance

POST :8080 /api/process-definitions/{key}/instances

PROCESS DEFINITION KEY

2251799813693524

VARIABLES JSON

```

{
  "orderId": "ORD-2041",
  "workObject": {
    "objectId": "OBJ-001",
    "type": "car",
    "material": "steel"
  }
}

```

Example payload for testing. Edit it to match the deployed process.

Start process instance

Started process instance

STARTED

PROCESS	VERSION
Process_0xxgmzy	6
DEFINITION KEY	INSTANCE KEY
2251799813693524	2251799813713872

Active User Tasks

GET :8090 /api/v1/camunda8/process-instances/{key}/user-tasks

Design Automobile

Activity_15oxmp3

Complete

(c) Active user-task completion

Figure 5.2: Frontend controls for deployment and process interaction

The dashboard in Figure 5.3 summarises the number of persisted records and detected violations and lists the latest activity-level results. The rows distinguish a calculation without a target, evaluations within their targets, and a violated evaluation.

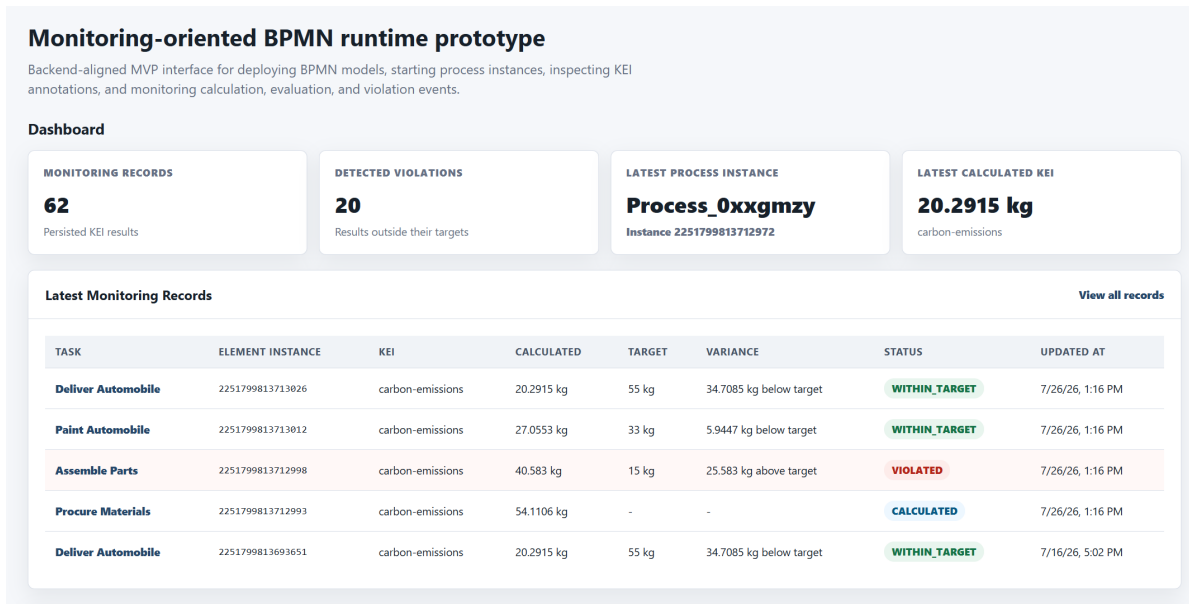


Figure 5.3: Dashboard summary and latest persisted monitoring records

Figure 5.4 presents the monitoring record breakdown for one process instance. It renders the BPMN diagram and places each recorded calculation on its corresponding monitored service task. Blue identifies a calculation without a target, green identifies a value within its target, and red identifies a violated target. Below the diagram, the table provides an activity-level breakdown for every monitored service task in the instance, including its element-instance identifier, calculated value, target, variance, timestamps, and status. Selecting an activity opens its expanded monitoring record with further process context, calculation provenance, resource usage, and technical identifiers, as illustrated for *Deliver Automobile* in Figure 5.5.

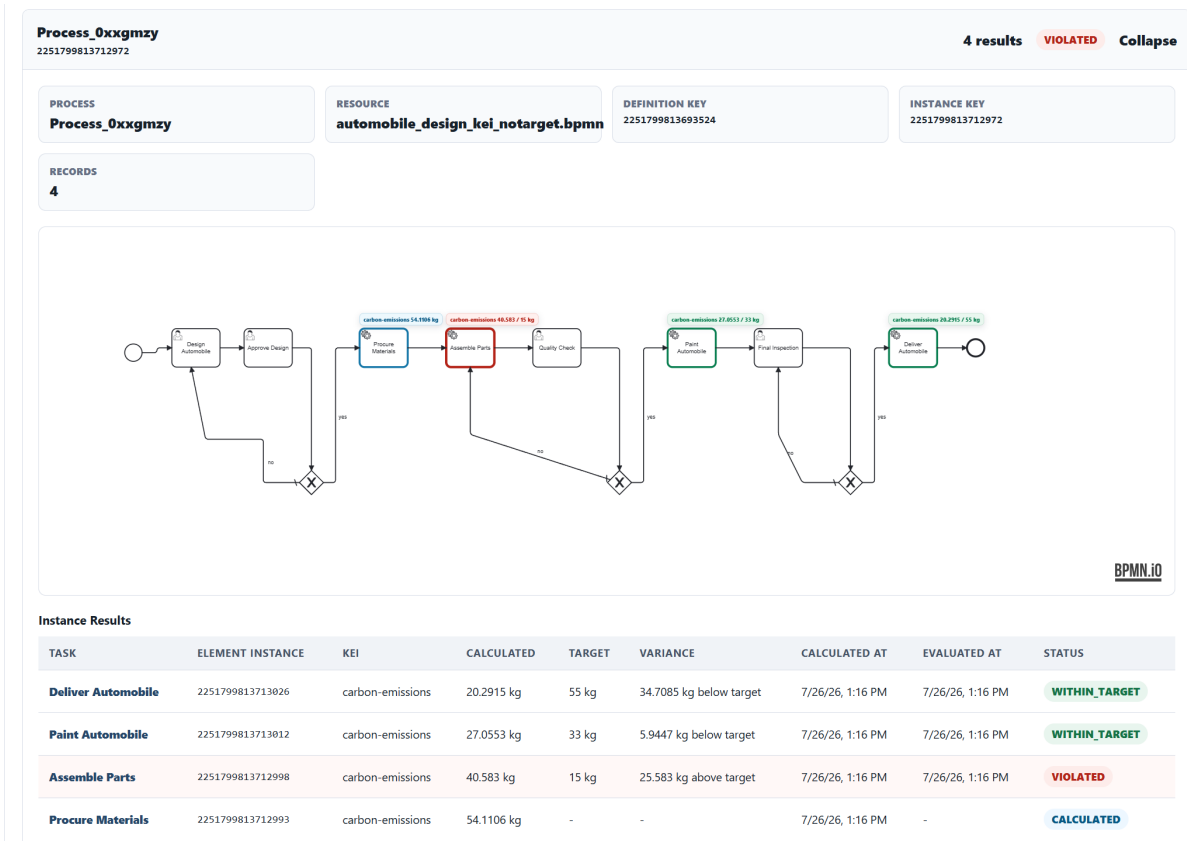


Figure 5.4: Process-instance inspection with BPMN element overlays

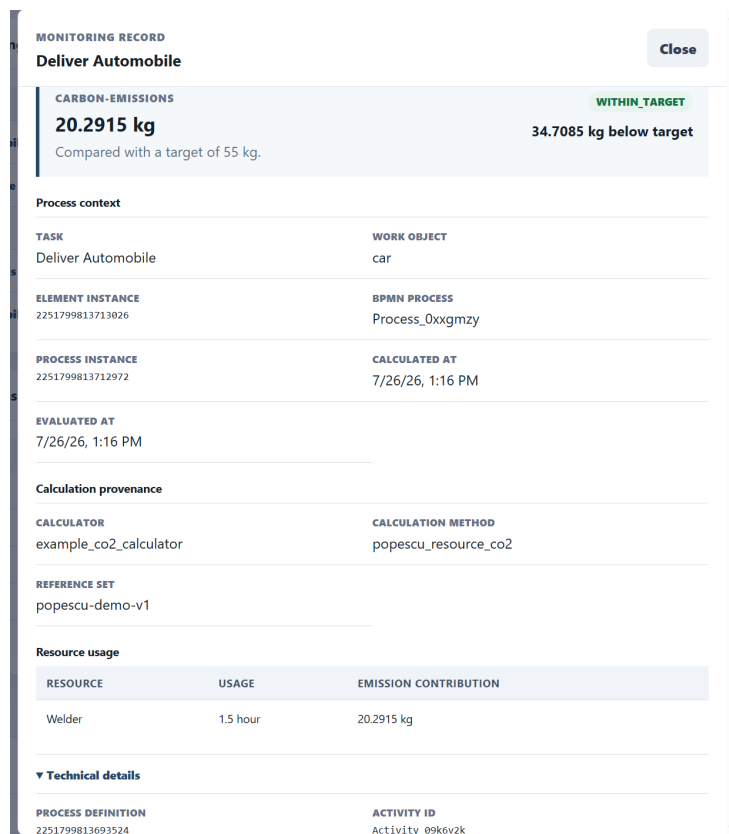


Figure 5.5: Expanded activity-level monitoring record

Figure 5.6 shows the dedicated violation query. Records can be filtered by process-definition key or BPMN process identifier, and each row presents the calculated value, target, variance, status, and occurrence time.

Active Violations

Current threshold violations GET :8094 /api/monitoring/violations/active

PROCESS DEFINITION KEY BPMN PROCESS ID

Apply filters

TASK	ELEMENT INSTANCE	KEI	CALCULATED	TARGET	VARIANCE	STATUS	OCCURRED AT
Assemble Parts	2251799813690966	carbon-emissions	40.583 kg	15 kg	25.583 kg above target	VIOLATED	7/15/26, 9:09 PM
Assemble Parts	2251799813690820	carbon-emissions	40.583 kg	15 kg	25.583 kg above target	VIOLATED	7/15/26, 8:52 PM
Assemble Parts	2251799813690237	carbon-emissions	20.2915 kg	15 kg	5.2915 kg above target	VIOLATED	7/15/26, 7:28 PM

Figure 5.6: Query and presentation of detected target violations

The connected live view in Figure 5.7 displays messages received from the calculation, evaluation, and violation topics. It presents a targetless calculation, two evaluations within target, and the evaluated violation together with its above-target difference.

Live Message Stream Socket connected

Calculation Events /topic/monitoring/calculations	Evaluation Events /topic/monitoring/evaluations	Violation Events /topic/monitoring/violations
Procure Materials Calculation KEI: carbon-emissions 54.1106 kg element instance 7/26/26, 1:16 PM 2251799813712993	Deliver Automobile Evaluation KEI: carbon-emissions 20.2915 kg WITHIN_TARGET element instance 7/26/26, 1:16 PM 2251799813713026	Assemble Parts Violation KEI: carbon-emissions 25.583 kg above target element instance 7/26/26, 1:16 PM 2251799813712998
	Paint Automobile Evaluation KEI: carbon-emissions 27.0553 kg WITHIN_TARGET element instance 7/26/26, 1:16 PM 2251799813713012	
	Assemble Parts Evaluation KEI: carbon-emissions 40.583 kg VIOLATED element instance 7/26/26, 1:16 PM 2251799813712998	

Figure 5.7: Live calculation, evaluation, and violation notifications

Chapter 6. Discussion

This chapter evaluates the realised prototype against the functional and quality requirements defined in Chapter 3. It then uses those findings to answer the research questions from Chapter 1. The final sections discuss the meaning of the findings, the limitations of the study and directions for future work.

6.1 Functional Requirement Evaluation

Table 6.1 evaluates every functional requirement using the three classifications defined in Section 3.8. A requirement is marked *satisfied* only when all behaviour in its statement is supported by the process execution, the outage and fault-injection assessments, or direct inspection of the realised call order and transformation. Requirements whose necessary assessment was not performed are marked *not evaluated*.

Table 6.1: Functional requirement evaluation

Requirement	Status	Evidence and assessment
FR-01	<i>Satisfied</i>	The annotated model was accepted by Process Registry and deployed by Camunda. Figure 5.2(a) shows the returned deployment key, process-definition key, process ID, version, and resource name.
FR-02	<i>Satisfied</i>	The deployed definition was started with the required process variables. Figure 5.2(b) shows the returned process-instance key and the started state.
FR-03	<i>Satisfied</i>	Listing 5.1 shows the KEI metadata in the submitted model. Process Registry invokes the parser before the deployment client, as shown in Listing 5.3. The registered representation and later monitoring records retain the corresponding KEI IDs, units, and optional targets.
FR-04	<i>Satisfied</i>	The deployment path registers the engine-assigned definition identifiers together with the original model, annotated elements, KEI ID, unit, and optional target. The later records contain the correct activity names and targets for the deployed definition, which confirms that this mapping was available during execution.
FR-05	<i>Satisfied</i>	The persisted records retain the process-definition, process-instance, element-instance, and BPMN-element identifiers received for the observed execution.
FR-06	<i>Satisfied</i>	Configured <code>default-worker</code> service tasks produced monitoring records containing the resource usage returned by Mock Operational Service.

Requirement	Status	Evidence and assessment
FR-07	<i>Satisfied</i>	Inspection of the worker implementation establishes that the completed task publisher is invoked after the operational response and before the Camunda job-completion command. The process instance advanced beyond the service tasks and the downstream records were produced.
FR-08	<i>Satisfied</i>	Observation retrieves annotations using the reported process definition and BPMN-element identifiers. It combines each returned annotation with the execution context and resource usage, as shown in Listing 5.6. The downstream records preserve both sources of data.
FR-09	<i>Satisfied</i>	Observation forms the route <code>kei.calculation.requested.carbon-emissions</code> . The configured CO2 queue successfully receives the calculation requests.
FR-10	<i>Satisfied</i>	The supported <code>carbon-emissions</code> requests were calculated for their corresponding service-task executions.
FR-11	<i>Satisfied</i>	The <i>Procure Materials</i> annotation has no target. Its completed calculation was persisted with status CALCULATED and is visible in Figures 5.3 and 5.7.
FR-12	<i>Not evaluated</i>	The automobile-assembly model uses only the <code>carbon-emissions KEI</code> and only the CO2 calculator has a bound route. No activity was executed with two distinct KEIs that both had working calculators.
FR-13	<i>Satisfied</i>	The process execution produced values below and above configured maximum targets. The two lower values were classified as WITHIN_TARGET, while <i>Assemble Parts</i> was classified as VIOLATED.
FR-14	<i>Satisfied</i>	Targetless calculation and completed evaluation paths both produced one activity-level record.
FR-15	<i>Satisfied</i>	Persisted results are available through Monitoring Results and are presented in a dashboard.
FR-16	<i>Satisfied</i>	The fault-injection assessment introduced missing KEI metadata, missing resource usage, an unroutable calculation request, a non-numeric target, and a handled calculation failure. Each condition produced the expected service-local diagnostic entry. No completed monitoring record was returned for any of the five injected cases.
FR-17	<i>Satisfied</i>	Figure 5.6 shows the queryable violation view and its filters. The violated <i>Assemble Parts</i> record is also visible through the dashboard, process-instance view, and violation live topic.
FR-18	<i>Satisfied</i>	Figure 5.4 shows the stored BPMN model with activity overlays and the records belonging to one process instance. Figure 5.5 shows the resource breakdown and detailed context available after selecting one activity.

Seventeen functional requirements are classified as *satisfied*. The multiple-KEI requirement is classified as *not evaluated*. The satisfied classifications establish the implemented deployment, registration, execution, calculation, evaluation, persistence, inspection, and failure-handling

paths examined in the evaluation.

6.2 Quality Requirement Evaluation

The quality requirements are evaluated through the response measures of their corresponding quality attribute scenarios (See Section 3.7). Architectural structure and source inspection can explain why a response is to be expected. On the other hand, only the prescribed experiment can provide the evidence required by a response measure.

Table 6.2: Quality requirement evaluation

Requirement	Status	Evidence and assessment
QR-01	<i>Satisfied</i>	Observation, CO2 Calculation, KEI Evaluation, and Monitoring Results were each stopped during a separate configured service-task execution. In every run, the worker completed the Camunda job once, the process instance reached its end event, and Camunda reported no workflow engine incident while the selected downstream service remained unavailable.
QR-02	<i>Not evaluated</i>	The implementation derives a routing key from the KEI ID and keeps the worker and registration schema independent of calculator code, which, in theory, facilitates the creation of other calculators. No second calculator was implemented, built, and run. No change set therefore exists that can satisfy the scenario's response measure.
QR-03	<i>Satisfied</i>	The five conditions named in the scenario were introduced separately. Observation identified missing metadata, missing resource usage, and the unroutable calculation request. KEI Evaluation identified the non-numeric target, while CO2 Calculation identified the handled failure and its cause. For every condition, the observed log entry identified the detecting component through its service-local logger and described the encountered condition.
QR-04	<i>Satisfied</i>	The process execution exercised every configured producer-consumer pair. Observation accepted the worker event. CO2 Calculation accepted the enriched request. KEI Evaluation and Monitoring Results accepted completed calculations. Monitoring Results accepted completed evaluations. The final records retain the execution identifiers, KEI data, calculated values, resource breakdown, and evaluation fields passed across those boundaries.

Downstream Fault Isolation, Failure Diagnosability, and Message Compatibility are classified as *satisfied*. Calculation Modifiability remains *not evaluated* because no additional calculator and corresponding change set exist.

6.3 Answers to the Research Questions

6.3.1 RQ1: Transforming BPMN4ES Metadata for Runtime Use

RQ1 asks how KEIs defined in BPMN4ES models can be transformed into a representation suitable for runtime interpretation in a BPM system.

The prototype answers this through two linked transformations. Before deployment, Process Registry reads the BPMN4ES XML extension and extracts the KEI ID, unit, optional target, icon, and the identity and type of the containing BPMN element. After deployment, it combines that modelling information with the process definition and deployment identifiers assigned by Camunda. Monitoring Results persists the combined representation.

At runtime, the process-definition key and BPMN-element ID reported by the worker select the registered annotations for the executed activity. Runtime components therefore receive a KEI representation without parsing the BPMN document again. Camunda executes the submitted model and supplies deployment and execution identifiers. It does not interpret the environmental extension.

The resulting answer is that BPMN4ES metadata can be made usable at runtime by parsing it outside the workflow engine, registering it against engine-assigned definition identifiers, and retrieving it with the definition and element identifiers emitted during execution. The deployment establishes this answer for the BPMN4ES elements handled by the parser and for Camunda 8.

6.3.2 RQ2: Providing Runtime Data from Process Execution

RQ2 asks how BPM process execution can provide the runtime data needed to monitor KEIs.

For configured service tasks, a Camunda job worker supplies the process-definition key, process-instance key, element-instance key, BPMN process ID, and BPMN-element ID. These identifiers state which execution occurred. The delegated operational call supplies the resource name, usage value, and unit for the completed work. The worker publishes both sets of facts in one completed-task event. Observation then adds the annotations registered for the same definition and element.

Camunda supplies execution identity, the operational boundary supplies resource-use facts, and the registered model supplies the KEI and target that apply to the activity. Observation joins these inputs before calculation.

The prototype therefore shows one practical way for process execution to support KEI monitoring without adding environmental semantics to the workflow engine. The answer is limited to configured service tasks handled by the selected worker. The resource-use values are read from `mock-execution-runs.json`, and no runtime observation is produced for user tasks, gateways, events, or service tasks assigned to another job type.

6.3.3 RQ3: Calculating KEIs from Runtime Data

RQ3 asks how KEI values can be calculated from process execution data and what data such calculations require.

The implemented carbon-emissions calculation requires the resource name, time used, and the unit in which that time is expressed. It also requires a resource profile containing the consumption rate, fuel type, and applicable units, together with an emission factor selected from the configured reference set.

For each resource, the calculator checks the units, combines the time used with the consumption rate and emission factor, and adds the resulting contributions. The KEI ID identifies the supported calculation. The model annotation and execution identifiers accompany the request so that the result can be evaluated and associated with the executed activity, but they do not enter the arithmetic.

The calculation method is adapted from Popescu [9]. The resource-use values and reference data are configured for the example, so the results demonstrate the calculation path without establishing the environmental accuracy of an industrial assembly process.

6.3.4 Main Research Question

The main research question asks how modelling-level KEIs can be used during process execution to support environmental monitoring and evaluation.

Modelling-level KEIs can be used at runtime by translating their BPMN4ES annotations into a registered representation associated with the identifiers assigned during deployment. The identifiers reported during execution then connect an observed service task and its operational facts to the applicable annotations. This supplies the inputs for calculation, optional target evaluation, persistence, and inspection.

The implemented approach allows the BPMN4ES extension to support the execution and monitoring phase of the Green BPM life cycle while fitting alongside the workflow engine's normal deployment and execution path. Camunda remains responsible for process execution, while the environmental meaning and processing remain in the monitoring system. Engine-specific interaction is confined to the integration and worker services. The prototype therefore demonstrates this separation for Camunda 8 and the implemented service-task path. Since no other engine was evaluated, it does not establish engine portability.

6.4 Limitations

6.4.1 Environmental Credibility

The prototype does not produce a credible estimate of emissions from an industrial automobile-assembly process. Its resource-use durations come from `mock-execution-runs.json`, while the Welder consumption rate, emission factor, and targets are configured for the example. No runtime resource-use value is measured from physical work.

The resource-based calculation inherits the limitations identified by Popescu [9]. Its result depends on complete and accurate resource data, and the implemented method covers atomic resources with a known average consumption rate. It does not allocate the consumption of shared resources among concurrent activities. The evaluator also treats the annotation unit and calculator result unit as compatible without validating or converting them.

6.4.2 Runtime Coverage

Runtime observation is limited to service tasks configured with the prototype's `default-worker` job type. Annotations on user tasks, gateways, events, or service tasks using another job type do not produce runtime observations. The worker does not obtain authoritative activity start and completion timestamps. The timestamps stored with monitoring records describe calculation and evaluation processing instead of the duration of the operational activity.

The worker still waits for the operational service and successful broker publication before completing its Camunda job. The demonstrated fault isolation applies only to the downstream monitoring services after publication. Monitoring records are not returned to the running process, so a detected violation cannot alter the process path.

6.4.3 Evaluation and Generalisation

The evaluation uses one automobile-assembly model, Camunda 8, one worker type, one KEI, one calculator, and one atomic resource profile. No second workflow engine adapter or calculator

was implemented. The study therefore does not demonstrate engine portability, calculation modifiability, multiple-KEI execution, or applicability to other environmental calculations.

The prototype was evaluated in a local deployment and was not integrated into an industrial environment. The requirements and prototype were produced within the same study without independent validation by industrial stakeholders. The outage and diagnostic assessments were performed manually, and no automated suite covers the complete deployment-to-monitoring path.

6.5 Future Work

6.5.1 Measured Operational Data and Lifecycle Timestamps

The first priority is to replace the durations stored in `mock-execution-runs.json` with data from the operational system responsible for the work being measured. Each value should retain its source, unit and measurement period.

Activity start and completion timestamps could be retrieved from Camunda's element-instance data using the element-instance key already carried by the worker. Execution listeners provide another possible lifecycle source [22]. Either approach should store the activity interval with the monitoring record and establish how that interval relates to the period covered by the operational measurement.

6.5.2 Additional Observation and Calculation Paths

A second workflow engine adapter would test whether the external mapping and execution-context contract are portable. Within Camunda, separate experiments with connectors, execution listeners, and exported element-instance data would establish the coverage and operational cost of each observation boundary.

A second KEI calculator should be implemented with representative inputs. An energy or waste calculator would reveal whether the current `timeUsed` field is sufficient or whether the request requires a typed quantity, observation interval, source, and domain-specific attributes. The resulting change set would provide the evidence required by the Calculation Modifiability scenario.

Further use cases should include shared resources and process-level aggregation. This would require an allocation method for assigning part of a shared resource's consumption to each activity, addressing a limitation already identified by Popescu [9].

6.5.3 Returning Monitoring Results to Process Execution

A future process model could use a monitoring result to select its subsequent path. This requires an explicit feedback point in the BPMN model. An intermediate message catch event could be placed after the monitored work [25], followed by an exclusive gateway or an activity that consumes the returned result.

The corresponding runtime interaction would be:

1. The process instance receives a unique monitoring correlation value as a process variable. The value is preserved through the monitoring events.
2. The process reaches the intermediate message catch event and waits for a monitoring result carrying the same correlation value.
3. A process-facing adapter receives the completed calculation or evaluation and sends a correlated message to Camunda.

4. Camunda stores selected result fields as process variables. Conditions on the following gateway use those variables to select the next activity.

The model and adapter require a shared contract that defines the returned variable names and maps fields such as the calculated value, target, difference, and evaluation status to them. The BPMN model must contain gateway expressions that use those names. The implementation must also handle a result that arrives before the process has created its message subscription.

An instance that waits at the catch event becomes dependent on the monitoring result. Models that do not require result-driven control can continue to use the current monitoring-only path.

6.5.4 Complete Quality Evaluation

Calculation Modifiability requires a working second calculator and inspection of the resulting changes. The messaging, outage, and diagnostic assessments should also be included in an automated integration suite covering the complete deployment-to-monitoring path.

6.6 Discussion Summary

The evaluation supports the main deployment-to-monitoring path for the Camunda model and carbon-emissions calculator used in the thesis. BPMN4ES annotations are transformed into a persistent runtime mapping, configured service-task executions provide correlation context and mock operational facts, and prototype services outside the workflow engine calculate, evaluate, persist, and expose activity-level results.

The environmental values remain demonstration outputs produced from data supplied by the prototype. Portability, calculator modifiability, lifecycle timing, broader BPMN coverage, and process feedback require further implementation and evaluation.

Chapter 7. Conclusion

The most important lesson arising from this work is that environmental monitoring must earn its place in process management. It should not require organisations to abandon the systems and processes on which their work already depends. It should fit around them, provide information that was previously unavailable, and leave ordinary process execution free to continue.

The work presented here is only a starting point. Its next development must place greater emphasis on the credibility of environmental calculations. Measured operational data, traceable reference sources and explicit uncertainty are necessary before calculated values can support real decisions. The range of supported situations must also expand. Environmental impact is rarely as predictable as one resource being used by one activity. Shared resources, indirect consumption, waste, external suppliers, changing emission factors, and effects that emerge across several activities all require different forms of observation, attribution, and calculation.

The framework should therefore become pluggable in every direction. Workflow engines, observation mechanisms, operational data sources, calculators, evaluation methods, and presentation tools should be replaceable or extendable through clear interfaces. This flexibility should not come at the cost of the businesses adopting it. Environmental monitoring should introduce as little disruption as possible and should never prevent an organisation from carrying out the work it was already able to perform.

Ease of use must remain equally important as the technical capability grows. Modellers need a practical way to state what should be monitored. Developers need understandable extension points. Operators need results whose meaning and origin they can inspect. A useful framework should give each of these groups leverage rather than transferring the system's complexity to them.

Environmental harm is easier to ignore when it remains invisible. Making it visible does not solve the problem by itself, but it creates a basis for questioning decisions and improving them. I hope this work encourages others to contribute to similar efforts, challenge their assumptions, and extend their reach. Being transparent about how our processes affect the world is a meaningful step towards treating that impact as something we are responsible for changing.

Bibliography

- [1] B. Gallotta, J. A. Garza-Reyes, A. Anosike, M. K. Lim, and I. Roberts, “A conceptual framework for the implementation of sustainability business processes,” en, [Online]. Available: <https://pomsmeetings.org/ConfProceedings/065/Full%20Papers/Final%20Full%20Papers/065-0877.pdf>.
- [2] F. Klessascheck, I. Weber, and L. Pufahl, “SOPA: A framework for sustainability-oriented process analysis and re-design in business process management,” *Information Systems and e-Business Management*, vol. 23, pp. 489–537, Jan. 2025. DOI: [10.1007/s10257-024-00695-x](https://doi.org/10.1007/s10257-024-00695-x).
- [3] V. van den Broek, “Going greener through bpm: A method for assessing processes environmental footprint and supporting continuous improvement,” Supervisors: Oktay Türetken and T. Tan, Master’s thesis, Eindhoven University of Technology, Eindhoven, The Netherlands, Aug. 2015. [Online]. Available: <https://research.tue.nl/en/studentTheses/going-greener-through-bpm/>.
- [4] J. Elkington, “Partnerships from *cannibals with forks: The triple bottom line of 21st-century business*,” en, *Environmental Quality Management*, vol. 8, no. 1, pp. 37–51, Sep. 1998, ISSN: 1088-1913, 1520-6483. DOI: [10.1002/tqem.3310080106](https://doi.org/10.1002/tqem.3310080106). [Online]. Available: <https://onlinelibrary.wiley.com/doi/10.1002/tqem.3310080106>.
- [5] M. Medema, B. Popescu, V. Andrikopoulos, and D. Karastoyanova, “A Life Cycle and Enabling Concepts for Green Business Process Management,” en, DOI: [10.1109/ICT4S68164.2025.00037](https://doi.org/10.1109/ICT4S68164.2025.00037).
- [6] A. Nowak, F. Leymann, D. Schleicher, D. Schumm, and S. Wagner, “Green business process patterns,” en, in *Proceedings of the 18th Conference on Pattern Languages of Programs*, Portland Oregon USA: ACM, Oct. 2011, pp. 1–10, ISBN: 978-1-4503-1283-7. DOI: [10.1145/2578903.2579144](https://doi.org/10.1145/2578903.2579144). [Online]. Available: <https://dl.acm.org/doi/10.1145/2578903.2579144>.
- [7] J. Recker, M. Rosemann, and E. R. Gohar, “Measuring the Carbon Footprint of Business Processes,” en, in *Business Process Management Workshops*, M. Zur Muehlen and J. Su, Eds., vol. 66, Series Title: Lecture Notes in Business Information Processing, Berlin, Heidelberg: Springer Berlin Heidelberg, 2011, pp. 511–520, ISBN: 978-3-642-20510-1 978-3-642-20511-8. DOI: [10.1007/978-3-642-20511-8_47](https://doi.org/10.1007/978-3-642-20511-8_47). [Online]. Available: http://link.springer.com/10.1007/978-3-642-20511-8_47.
- [8] A. Nowak, F. Leymann, and D. Schumm, “The Differences and Commonalities between Green and Conventional Business Process Management,” Dec. 2011, pp. 569–576. DOI: [10.1109/DASC.2011.105](https://doi.org/10.1109/DASC.2011.105).
- [9] B.-P. Popescu, “Environmental Sustainability Calculator Service for Business Processes,” en, [Online]. Available: <https://fse.studenttheses.ub.rug.nl/id/eprint/34080>.

- [10] S. Seidel, J. Recker, and J. Vom Brocke, “Green Business Process Management,” en, in *Green Business Process Management*, J. Vom Brocke, S. Seidel, and J. Recker, Eds., Berlin, Heidelberg: Springer Berlin Heidelberg, 2012, pp. 3–13, ISBN: 978-3-642-27487-9 978-3-642-27488-6. DOI: [10.1007/978-3-642-27488-6_1](https://doi.org/10.1007/978-3-642-27488-6_1). [Online]. Available: https://link.springer.com/10.1007/978-3-642-27488-6_1.
- [11] L. Willmann, *Implementing sustainability of workflows into a conventional bpm engine*, English, Bachelor’s thesis, Supervisors: M. Medema and D. Karastoyanova, 2025. [Online]. Available: <https://fse.studenttheses.ub.rug.nl/id/eprint/36365>.
- [12] A. Nowak, T. Binz, C. Fehling, O. Kopp, F. Leymann, and S. Wagner, “Pattern-driven green adaptation of process-based applications and their runtime infrastructure,” en, *Computing*, vol. 94, no. 6, pp. 463–487, Jun. 2012, ISSN: 1436-5057. DOI: [10.1007/s00607-012-0188-x](https://doi.org/10.1007/s00607-012-0188-x). [Online]. Available: <https://doi.org/10.1007/s00607-012-0188-x>.
- [13] D. Schäfer, F. Klessascheck, T. Kampik, and L. Pufahl, “Can We Leverage Process Data from ERP Systems for Business Process Sustainability Analyses?” In Mar. 2025, ISBN: 978-3-031-82224-7. DOI: [10.1007/978-3-031-82225-4_56](https://doi.org/10.1007/978-3-031-82225-4_56).
- [14] M. Medema, *Bpmn4es: An extension of bpmn-js for modelling business processes using the bpmn4es notation*, 2025. [Online]. Available: <https://github.com/michel-medema/BPMN4ES>.
- [15] J. Rasmussen, *Implementing sustainability of workflows into a conventional bpmn workflow engine*, English, Bachelor’s thesis, Supervisors: M. Medema and D. Karastoyanova, 2015. [Online]. Available: <https://fse.studenttheses.ub.rug.nl/id/eprint/37025>.
- [16] Camunda, *Java client: Getting started*, 2024. [Online]. Available: <https://docs.camunda.io/docs/apis-tools/java-client/getting-started/>.
- [17] Camunda, *Migrate from grpc to the orchestration cluster api*, 2024. [Online]. Available: <https://docs.camunda.io/docs/apis-tools/migration-manuals/migrate-from-grpc-to-orchestration-cluster-api/>.
- [18] Camunda, *8.8 release notes: Apis & tools*, 2024. [Online]. Available: <https://docs.camunda.io/docs/reference/announcements-release-notes/880/880-release-notes/#apis--tools>.
- [19] Camunda, *Service tasks*, 2024. [Online]. Available: <https://docs.camunda.io/docs/components/modeler/bpmn/service-tasks/>.
- [20] Camunda, *Job workers*, 2024. [Online]. Available: <https://docs.camunda.io/docs/components/concepts/job-workers/>.
- [21] Camunda, *Outbound connectors vs. job workers*, 2024. [Online]. Available: <https://docs.camunda.io/docs/components/concepts/outbound-connectors-job-workers/>.
- [22] Camunda, *Execution listeners*, 2024. [Online]. Available: <https://docs.camunda.io/docs/components/concepts/execution-listeners/>.
- [23] Camunda, *User task listeners*, 2024. [Online]. Available: <https://docs.camunda.io/docs/components/concepts/user-task-listeners/>.
- [24] L. Bass, P. Clements, and R. Kazman, *Software Architecture in Practice*, en. Harlow: Pearson Education, 2013, ISBN: 978-0-321-81573-6 978-0-13-294277-5.
- [25] Camunda, *Message events*, 2024. [Online]. Available: <https://docs.camunda.io/docs/components/modeler/bpmn/message-events/>.

Glossary

BPMN4ES An extension of BPMN that attaches KEIs and optional target values to process-model elements. 1, 2, 6, 10, 13, 15, 17, 26, 27, 30, 35, 37, 46, 47, 64, 65, 67

detected violation A persisted monitoring record whose evaluation status is VIOLATED. It indicates that a calculated value exceeded its configured target and does not by itself denote regulatory non-compliance. 1, 16, 19, 22, 32, 35, 39, 57, 65

emission factor A coefficient used to convert activity or resource consumption data into an estimated amount of emitted greenhouse gas. 5, 11–14, 16, 30, 35, 52, 54, 64, 65, 68

monitoring record An activity-level persisted record created from a completed environmental calculation when no target is defined, or from the corresponding completed target evaluation when a target is defined. It links the result to its process execution, BPMN element, KEI annotation, calculation data, and optional evaluation outcome. 1, 3, 6, 15, 16, 18, 19, 22, 24, 30, 32, 33, 35, 39, 43, 44, 54, 56, 58, 59, 61, 62, 65, 66, 71

runtime mapping An execution-time representation that associates deployed process identifiers and BPMN element identifiers with environmental metadata. 18, 26, 35, 37, 39, 67

workflow engine A software system responsible for executing process instances according to an executable process model. 1, 6, 12–19, 21, 23, 24, 26–28, 35, 37, 39, 46, 63–67

Acronyms

ADD architectural design decision. 1, 26–32

API application programming interface. 18, 19, 35, 37

BPM Business process management. 3, 5–9, 12, 64

BPMN Business Process Model and Notation. 1, 3, 5, 6, 8, 10, 12–16, 18, 19, 21, 26–29, 35, 37, 39, 43, 46–49, 54, 55, 58–62, 64, 66, 67, 71

DOM Document Object Model. 37

GHG greenhouse gas. 5

Green BPM Green business process management. 1, 5–10, 12, 13, 15, 65

HTTP Hypertext Transfer Protocol. 44, 45

JAR Java archive. 45

JPA Java Persistence API. 45

JSON JavaScript Object Notation. 16, 48, 50

KEI key environmental indicator. 1, 2, 5–8, 10, 12–19, 21–24, 26, 27, 29–32, 35, 37, 39, 41, 43, 44, 46, 47, 49–52, 54, 61–66, 71

KPI key performance indicator. 5, 7

REST Representational State Transfer. 35, 37, 43, 45, 54, 55

SQL Structured Query Language. 45

STOMP Simple Text Oriented Messaging Protocol. 43, 45, 54, 55

URI Uniform Resource Identifier. 46

XML Extensible Markup Language. 10, 37, 45, 64